Caso de estudio

Mariana Belén Cruz Rodríguez Yofre H. García Gómez

Tabla de contenidos

1 Introducción **5**

2 Objetivos **6**
2.1 Objetivo General 6
2.2 Objetivos Específicos 6

3 Preliminares **7**
3.1 Contexto histórico y tecnológico 7
3.1.1 El alfabeto náhuatl y la preservación del patrimonio documental . . . 7
3.1.2 El reto del reconocimiento óptico de caracteres en lenguas indígenas . 7
3.2 Planteamiento del Problema 11
3.2.1 Limitaciones de los enfoques tradicionales en glifos con alta variabilidad morfológica 11
3.2.2 La dicotomía matemática: Modelos Generativos Secuenciales (HMM) vs. Modelos Discriminativos Tensoriales (CNN) 11
3.3 Justificación desde las Matemáticas Aplicadas 12
3.3.1 Inferencia estocástica vs. Optimización en espacios tensoriales 12
3.3.2 El tratamiento de la dimensionalidad y la invarianza geométrica . . . 13
3.3.3 Interpretabilidad matemática y escasez de datos 13

4 Fundamentos teóricos: Cadenas de Markov a Tiempo Continuo y Modelos Ocultos **15**
4.1 Espacio Probabilístico y Procesos Estocásticos Subyacentes 15
4.2 Cadenas de Markov a Tiempo Continuo (CTMC) 16
4.2.1 Dinámica Infinitesimal y Generador de la Cadena 18
4.3 Utilidad hacia Cadenas de Márkov Ocultas (HMM) 19
4.4 Inferencia y Filtrado en el Modelo Oculto 20
4.4.1 Ejemplo ilustrativo del algoritmo Forward–Backward 22
4.4.2 Cálculo Forward 23
4.5 Estimación de Parámetros y Calibración del Modelo 26

5 Fundamentos matemáticos de las Redes Neuronales Convolucionales **28**
5.1 Fundamentos CNN 28
5.2 Formulación Matemática de una CNN 28
5.3 Representación Tensorial de Imágenes 28
5.3.1 Estructura Funcional de la Red 29
5.3.2 Parámetros del Modelo 29
5.3.3 Kernels Convolucionales 30
5.4 Operación de Convolución 32
5.4.1 Caso Unicanal 32

5.4.2 Convolución Multicanal . . . 35
5.4.3 Observación estructural . . . 39
5.5 Stride y Padding . . . 39
5.5.1 Stride . . . 39
5.5.2 Padding . . . 41
5.5.3 Función de activación . . . 43
5.5.4 Pooling . . . 45
5.5.5 Capas completamente conectadas . . . 47
5.5.6 Aplicación de la función de activación . . . 48
5.5.7 Función de pérdida . . . 49
5.5.8 Entrenamiento . . . 50
5.5.9 Método del descenso del gradiente . . . 50
5.5.10 Dropout . . . 55
5.5.11 Optimización . . . 58

6 Base de datos **60**
6.1 Generación y Caracterización del Conjunto de Datos . . . 60
6.1.1 Construcción del Conjunto de Datos . . . 60
6.1.2 Inexistencia de datasets públicos y justificación de la generación sintética 60
6.1.3 Diseño de las plantillas base y morfología del glifo náhuatl . . . 60
6.2 Transformaciones Geométricas y Estocásticas (Data Augmentation) . . . 61
6.2.1 Aplicación de grupos de Lie en transformaciones afines . . . 61
6.2.2 Modelado de ruido y degradación . . . 61
6.2.3 Análisis estadístico del dataset resultante . . . 61
6.3 Mapeo Dimensional: De $\mathbb{R}^{28\times28}$ a $\mathbb{R}^{T}$ . . . 62
6.3.1 Definición del operador de ventana deslizante vertical . . . 62
6.4 Extracción Matemática de Rasgos Estructurales (Feature Engineering) . . . 62
6.4.1 Cálculo del Centro de Gravedad . . . 62
6.4.2 Transiciones de estado y cruces por cero . . . 62
6.4.3 Perfiles superior e inferior (Envoltura convexa) . . . 63
6.5 Cuantificación y Discretización del Espacio de Observaciones . . . 63
6.5.1 Función de cuantificación por intervalos (Binning) . . . 63
6.5.2 Codificación binaria y construcción del alfabeto de 256 símbolos observables . . . 63
6.5.3 Análisis de la entropía de Shannon de las secuencias generadas . . . 63
6.6 Construcción del conjunto de datos . . . 64
6.6.1 Generación de las muestras . . . 65
6.7 Carga y organización de los datos . . . 66
6.8 Normalización y aumentación de datos . . . 66

7 Caso de estudio **68**
7.1 CASO CNN . . . 68
7.1.1 Arquitectura de la red neuronal convolucional . . . 68
7.1.2 Primer bloque convolucional . . . 68
7.1.3 Segundo y tercer bloque convolucional . . . 69
7.1.4 Clasificador . . . 69

7.2 Entrenamiento 70
7.3 Sistema de inferencia interactivo 70
7.4 CASO HMM 71
7.5 CASO HMM 71
7.5.1 Configuración del entorno experimental 71
7.5.2 Definición de parámetros del experimento 72
7.5.3 Obtención del conjunto de datos 73
7.5.4 Arquitectura del HMM 73
7.5.5 Preprocesamiento de imágenes 73
7.5.6 Extracción de características 75
7.5.7 Construcción de los conjuntos de entrenamiento y prueba 76
7.5.8 Entrenamiento de los modelos 76
7.5.9 Inferencia y clasificación 77
7.5.10 Evaluación del desempeño 78

8 Conclusiones: Comparación entre HMM y CNN en Reconocimiento de Caracteres 79
8.1 Conclusiones 79

9 Referencias 80

1 Introducción

(pendiente)

2 Objetivos

2.1. Objetivo General

Comparar el desempeño de los Modelos Ocultos de Markov (HMM) con esquema de remuestreo *bootstrap* y las Redes Neuronales Convolucionales (CNN) en la clasificación de imágenes monocromáticas de glifos del alfabeto náhuatl.

2.2. Objetivos Específicos

1. **Preprocesar y caracterizar el conjunto de datos** de imágenes monocromáticas de glifos del alfabeto náhuatl, aplicando técnicas de normalización, aumentación geométrica y generación de ruido controlado, para establecer una base de evaluación estandarizada y robusta que permita la comparación equitativa de ambos modelos.
2. **Implementar y ajustar un modelo de clasificación basado en Modelos Ocultos de Markov (HMM)** categóricos, integrado con un esquema de remuestreo *bootstrap*. El propósito es modelar la secuencia de características estructurales extraídas de los glifos y cuantificar la incertidumbre estadística en su identificación.
3. **Diseñar y entrenar una arquitectura de Red Neuronal Convolucional (CNN)** optimizada para imágenes de baja resolución, formulando adecuadamente la función de pérdida y el proceso de optimización (ej. Adam), para extraer características espaciales jerárquicas y maximizar la precisión en la clasificación de las distintas clases de glifos.
4. **Evaluar y comparar el rendimiento de ambos enfoques** (HMM y CNN) mediante métricas de desempeño (exactitud, precisión, exhaustividad y función de pérdida), tiempos de convergencia y requerimientos de recursos computacionales, para identificar las ventajas, limitaciones y el ámbito de aplicabilidad de cada método en el contexto del reconocimiento de patrones glíficos.

3 Preliminares

3.1. Contexto histórico y tecnológico

3.1.1. El alfabeto náhuatl y la preservación del patrimonio documental

El náhuatl es una de las lenguas más importantes de Mesoamérica, con una historia escrita que se remonta a siglos antes de la llegada de los españoles. Originalmes nahuas (llamados *tlacuilos*) utilizaban un sistema de glifos y pictogramas para registrar información histórica, tributaria y ceremonial en códices. Tras la conquista, se desarrolló una adaptación del alfabeto latino que incorporaba convenciones ortográficas específicas para representar los sonidos propios del náhuatl.

En la actualidad, han surgido propuestas de **sistemas de escritura intermedios** que buscan establecer un puente entre la tradición glífica ancestral y la escritura alfabética moderna. Un ejemplo es el siguiente abecedario provisional Náhuatl (ver @abcnauatl), el cual asigna símbolos geométricos simplificados a los fonemas de la lengua, manteniendo cierta estética visual mesoamericana mientras facilita la transición hacia la lectoescritura.

La preservación de estos documentos y sistemas de escritura no solo implica su conservación física, sino también su digitalización y análisis computacional para garantizar su accesibilidad a futuras generaciones. Sin embargo, este proceso enfrenta un desafío importante: la transcripción manual por parte de especialistas es un proceso lento, costoso y propenso a errores. En este contexto, el desarrollo de herramientas automatizadas de reconocimiento se vuelve crucial.

No obstante, a diferencia de los textos alfabéticos modernos, la naturaleza particular de estos sistemas de escritura representa un desafío de modelado matemático y computacional que requiere técnicas avanzadas de reconocimiento de patrones.

3.1.2. El reto del reconocimiento óptico de caracteres en lenguas indígenas

El Reconocimiento Óptico de Caracteres (OCR, por sus siglas en inglés) ha alcanzado niveles de madurez notables para la digitalización de textos en lenguas mayoritarias con alfabetos estandarizados, como el español o el inglés. Sin embargo, su aplicación a sistemas de escritura indígenas, y particularmente a propuestas como el Abecedario Provisional Náhuatl, enfrenta desafíos que trascienden la ingeniería de software y se adentran en el aprendizaje automático. Algunos de estos desafíos son:



Figura 3.1: Construcción de un “alfabeto” fónico para el náuatl.

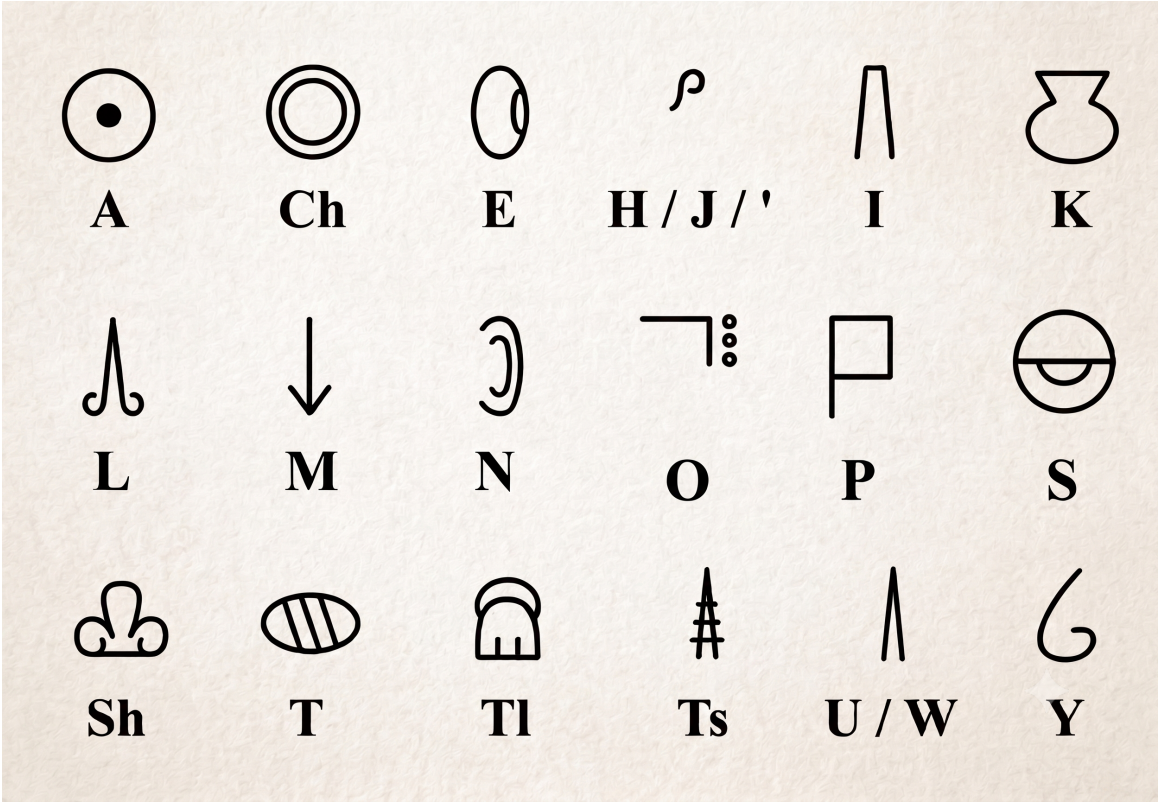


Figura 3.2: Representación del alfabeto fónico háhuatl.

1. **Variabilidad en la representación fonética:** A diferencia de un alfabeto estandarizado donde cada sonido tiene una representación única, en el náhuatl existe cierta flexibilidad en la elección del glifo para representar un fonema. Por ejemplo, para el sonido /e/, teóricamente se podría elegir entre el glifo de *etl* (frijol) o *elotl* (elote). Esta ambigüedad en el diseño ortográfico representa un desafío para la estandarización.
2. **Falta de estandarización tipográfica:** No existe un "tipo de letra" único para estos sistemas. Cada material educativo, comunidad o institución puede emplear ligeras variantes de los símbolos, lo que genera una alta variabilidad en las formas geométricas que un sistema de reconocimiento debe procesar.
3. **Ausencia de conjuntos de datos públicos:** A diferencia de alfabetos como el latino, para los cuales existen repositorios masivos y estandarizados (como EMNIST o MNIST), no existen bases de datos públicas de glifos náhuatl etiquetados. Esto limita severamente la aplicación directa de técnicas modernas de aprendizaje profundo que requieren grandes volúmenes de datos.
4. **Variabilidad morfológica en el trazo:** Una vez establecida la elección del glifo para un fonema (por ejemplo, decidir que /e/ se representa con el glifo de "frijol"), el sistema debe ser capaz de reconocer dicho glifo independientemente de las variaciones en el trazo, la escala, la rotación o pequeñas deformaciones que introduzca el usuario al dibujarlo.

Delimitación del problema de investigación:

Para abordar este problema desde las matemáticas aplicadas, es necesario realizar una delimitación clara del alcance. Si bien la variabilidad en la elección del pictograma (frijol vs. elote) es un problema lingüístico relevante, su resolución excede el ámbito de esta tesis.

Por lo tanto, **este trabajo asume un sistema ortográfico fijo y estandarizado**: el Abecedario Provisional Náhuatl mostrado en la Figura Figura 3.2, donde se ha establecido de manera definitiva qué glifo representa a cada fonema. Bajo esta suposición, el problema se traslada al terreno del **reconocimiento de patrones**, el cual presenta desafíos matemáticos propios que justifican la comparación de dos enfoques fundamentales:

- **Modelos Ocultos de Markov (HMM):** Que modelan la secuencia estocástica de los rasgos geométricos extraídos del trazo mediante procesos probabilísticos.
- **Redes Neuronales Convolucionales (CNN):** Que aprenden las invariantes espaciales y topológicas directamente de la matriz de píxeles mediante optimización en espacios tensoriales.

La comparación sistemática de estos dos enfoques permitirá determinar qué arquitectura matemática es más robusta para reconocer los glifos de este sistema provisional, sentando las bases para futuras herramientas de digitalización del patrimonio lingüístico náhuatl.

3.2. Planteamiento del Problema

El reconocimiento automático de patrones en sistemas de escritura no estandarizados presenta desafíos que exceden las capacidades de los algoritmos de reconocimiento óptico de caracteres (OCR) convencionales. Al centrar el estudio en el "alfabeto Náhuatl", el problema de investigación se formula desde la perspectiva de la modelación matemática y el aprendizaje automático, identificando dos grandes áreas de complejidad: la naturaleza morfológica de los glifos y la dicotomía entre los paradigmas de modelado estocástico y tensorial.

3.2.1. Limitaciones de los enfoques tradicionales en glifos con alta variabilidad morfológica

Los sistemas de OCR tradicionales y los modelos de clasificación de imágenes estándar asumen la existencia de una tipografía subyacente o, al menos, una geometría rígida y estandarizada para cada clase (carácter). En el caso del alfabeto latino, la varianza entre distintas instancias de la misma letra es relativamente baja y puede ser modelada mediante transformaciones afines simples.

Sin embargo, los glifos del sistema provisional náhuatl poseen una naturaleza geométrica y pictográfica que rompe estos supuestos. Al no existir una fuente tipográfica digital oficial, cada instancia de un glifo (por ejemplo, el símbolo correspondiente a la sílaba "Tl") presenta variaciones significativas en la curvatura, la proporción de sus componentes y la continuidad de sus trazos.

Matemáticamente, esto se traduce en distribuciones de probabilidad de las características de la imagen con una varianza intra-clase extremadamente alta y, en muchos casos, no gaussiana. Los enfoques tradicionales, que dependen de la correlación de plantillas (template matching) o de la extracción manual de características geométricas rígidas, fallan al no poder generalizar sobre esta alta dispersión morfológica. Además, la degradación natural de los documentos físicos (ruido de alta frecuencia, manchas, pérdida de contraste) introduce incertidumbre en los bordes y contornos de los glifos, lo que exige modelos matemáticos intrínsecamente robustos al ruido y a las deformaciones locales.

3.2.2. La dicotomía matemática: Modelos Generativos Secuenciales (HMM) vs. Modelos Discriminativos Tensoriales (CNN)

Para abordar la alta variabilidad morfológica descrita anteriormente, la literatura de visión por computadora y reconocimiento de patrones ofrece dos paradigmas matemáticos fundamentalmente distintos, cada uno con fortalezas y limitaciones teóricas particulares:

1. El enfoque estocástico y secuencial (Modelos Ocultos de Markov - HMM): Los HMM modelan el glifo no como una imagen estática, sino como la realización observable de un proceso estocástico secuencial. Al proyectar la imagen 2D en una secuencia 1D de características estructurales (como la distribución de "tinta", el centro de gravedad o las transiciones de bordes), el HMM utiliza matrices de transición y de emisión para capturar la

probabilidad de evolución del trazo. *Ventaja matemática:* Poseen una sólida fundamentación en la teoría de la probabilidad, permiten cuantificar la incertidumbre y son altamente interpretables (es posible analizar las probabilidades de transición entre estados). *Limitación:* Dependen críticamente de la *ingeniería manual de características*. La proyección de 2D a 1D conlleva una pérdida irreversible de información topológica y espacial, y la matriz de emisión sufre la "maldición de la dimensionalidad" si el alfabeto de símbolos observables es demasiado grande.

2. El enfoque de optimización en espacios tensoriales (Redes Neuronales Convolucionales - CNN): Las CNN abordan el problema desde el análisis funcional y el álgebra lineal en espacios de alta dimensión. Al tratar la imagen como un tensor en $\mathbb{R}^{H\times W\times C}$, aplican operadores de convolución (productos internos locales) que aprenden de manera endógena (automática) filtros detectores de características jerárquicas. *Ventaja matemática:* Gracias a la compartición de parámetros y las operaciones de *pooling*, son naturalmente equivariantes a traslaciones y robustas a pequeñas deformaciones geométricas, eliminando la necesidad de diseñar características manuales. *Limitación:* Funcionan como modelos "caja negra" (baja interpretabilidad matemática de sus decisiones internas) y requieren una gran cantidad de datos para ajustar sus miles de parámetros mediante descenso de gradiente, evitando el sobreajuste.

Formulación del problema de investigación: Dada la escasez de datos etiquetados para lenguas indígenas (lo que penaliza a las CNN) y la complejidad geométrica de los glifos (lo que penaliza a los HMM por la pérdida de información espacial en la extracción de rasgos), no existe un consenso teórico sobre qué paradigma es óptimo para este dominio específico.

Por lo tanto, el problema central de esta tesis es determinar, mediante un protocolo experimental riguroso, **cuál de estos dos marcos matemáticos (la inferencia probabilística secuencial (HMM) o la optimización de funciones no lineales en tensores (CNN)) ofrece un mejor equilibrio entre precisión de clasificación, eficiencia computacional e interpretabilidad** para el reconocimiento de los glifos del alfabeto náhuatl, y bajo qué condiciones morfológicas cada modelo demuestra su superioridad.

3.3. Justificación desde las Matemáticas Aplicadas

La elección de comparar Modelos Ocultos de Markov (HMM) y Redes Neuronales Convolucionales (CNN) para el reconocimiento de los glifos del alfabeto náhuatl no responde únicamente a tendencias tecnológicas, sino a la necesidad de evaluar dos paradigmas matemáticos fundamentalmente distintos para abordar un problema de clasificación de patrones con alta variabilidad morfológica. Desde la perspectiva de las matemáticas aplicadas, esta investigación se justifica a través de tres ejes teóricos principales:

3.3.1. Inferencia estocástica vs. Optimización en espacios tensoriales

El primer paradigma, representado por los HMM, aborda el reconocimiento de patrones desde la **teoría de la probabilidad y los procesos estocásticos**. En este enfoque, el glifo no se

trata como un objeto geométrico estático, sino como la realización observable de un proceso secuencial. La inferencia del carácter se formula como un problema de máxima verosimilitud, donde el algoritmo de *Expectation-Maximization* (Baum-Welch) estima los parámetros de transición y emisión en un espacio de estados discretos. La fortaleza matemática de este modelo radica en su rigor probabilístico y su capacidad para cuantificar la incertidumbre inherente a los trazos degradados o ambiguos.

El segundo paradigma, representado por las CNN, se fundamenta en el **análisis funcional y la optimización no lineal en espacios de alta dimensión**. Aquí, la imagen se modela como un tensor en $\mathbb{R}^{H\times W\times C}$, y el reconocimiento se formula como la minimización de una función de pérdida (como la entropía cruzada) mediante el flujo de gradiente (descenso de gradiente estocástico). La justificación de este enfoque reside en su capacidad para aproximar funciones no lineales complejas y aprender operadores de convolución que actúan como detectores de características locales, sin requerir una modelación explícita de la dinámica subyacente del trazo.

Comparar ambos enfoques permite determinar si, para un sistema de escritura con alta varianza intra-clase, es más eficiente modelar la **dependencia secuencial de los rasgos** (HMM) o aprender la **invarianza topológica mediante composiciones de funciones no lineales** (CNN).

3.3.2. El tratamiento de la dimensionalidad y la invarianza geométrica

Un aspecto crucial en el reconocimiento de los glifos náhuatl es cómo cada modelo matemático gestiona la dimensionalidad de la entrada y las transformaciones geométricas.

Los HMM requieren una proyección del espacio bidimensional de la imagen ($\mathbb{R}^{28\times 28}$) a una secuencia unidimensional de características discretas ($\mathbb{R}^{T}$). Matemáticamente, esta reducción de dimensionalidad conlleva una pérdida irreversible de información espacial y topológica, obligando al modelo a depender críticamente de la ingeniería manual de características (como el cálculo de momentos, perfiles y transiciones).

Por el contrario, las CNN operan directamente sobre el tensor de la imagen, preservando la estructura espacial local. Gracias a la propiedad de equivarianza por traslación inducida por la compartición de parámetros en los kernels convolucionales, y a la invarianza aproximada proporcionada por las operaciones de *pooling*, las CNN manejan las deformaciones geométricas (rotaciones, traslaciones y escalados) de manera endógena. Evaluar el impacto de estas dos estrategias de manejo de la dimensionalidad es esencial para entender los límites de cada modelo frente a la falta de estandarización tipográfica de los glifos.

3.3.3. Interpretabilidad matemática y escasez de datos

Finalmente, la investigación se justifica por la naturaleza del conjunto de datos. Al tratarse de un sistema de escritura indígena, no existen repositorios masivos de datos etiquetados, lo que impone un escenario de **aprendizaje con datos limitados**.

Los modelos probabilísticos como los HMM, al tener un espacio de parámetros mucho más restringido (matrices de transición y emisión) y estar guiados por supuestos estructurales fuertes (como la topología de Bakis para secuencias unidireccionales), suelen ser más robustos ante la escasez de datos y ofrecen una **interpretabilidad matemática directa**: es posible analizar las probabilidades de transición entre estados para entender cómo el modelo "decide" la clase.

En contraste, las CNN poseen miles de parámetros ajustables, lo que las hace propensas al sobreajuste en escenarios de pocos datos, requiriendo técnicas de regularización (como *Dropout* o *Weight Decay*) y aumentación de datos sintéticos. Además, su naturaleza de "caja negra" dificulta la interpretación de las decisiones internas.

Por lo tanto, esta tesis aporta valor a las matemáticas aplicadas al cuantificar formalmente el equilibrio entre **precisión, costo computacional, robustez ante datos limitados e interpretabilidad**, estableciendo un precedente metodológico para el procesamiento computacional de lenguas originarias y sistemas de escritura no estandarizados.

4 Fundamentos teóricos: Cadenas de Markov a Tiempo Continuo y Modelos Ocultos

4.1. Espacio Probabilístico y Procesos Estocásticos Subyacentes

Para establecer formalmente una Cadena de Markov a Tiempo Continuo (CTMC), es necesario partir de los axiomas fundamentales de la teoría de la medida, para más detalle consultar el libro de @Rudin1987.

Para esto, se asumirá que el lector tiene conocimientos previos sobre teoría de la medida como lo son las σ-álgebras, espacio muestral y medida de probabilidad, entre otros.

Espacio de probabilidad filtrado

Sea $(\Omega, \mathcal{F}, \mathbb{P})$ un espacio de probabilidad, donde Ω es el espacio muestral, $\mathcal{F}$ es una σ-álgebra sobre Ω, y $\mathbb{P}$ es una medida de probabilidad. Sea $\mathbb{F} = \{\mathcal{F}_t\}_{t\geq 0}$ una filtración, es decir, una familia no decreciente de sub-σ-álgebras de $\mathcal{F}$ tal que $\mathcal{F}_s \subseteq \mathcal{F}_t \subseteq \mathcal{F}$ para todo $0 \leq s < t$. El conjunto $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ se denomina espacio de probabilidad filtrado.

Espacio de estados discreto

Definimos el espacio de estados S como un conjunto discreto y finito (o numerable).

La topología en S es discreta y su σ-álgebra asociada es el conjunto potencia $\mathcal{P}(S)$.

En el contexto de nuestro proyecto, S representa el conjunto de posibles colores o intensidades de un pixel (por ejemplo, el espacio RGB donde $|S| = 256^3$, o una escala de grises finita).

Proceso estocástico a Tiempo Continuo

Un proceso estocástico en tiempo continuo con espacio de estados discreto S es una familia de variables aleatorias

$$X = \{X(t) : t \in \mathbb{R}_{\geq 0}\},$$

donde cada $X(t) : \Omega \to S$ es una función medible respecto a $\mathcal{F}_t$.

Interpretamos $X(t)$ como el "color verdadero" del pixel en el instante (o parámetro continuo) t.

4.2. Cadenas de Markov a Tiempo Continuo (CTMC)

La transición del concepto general de proceso estocástico al de proceso de Markov requiere la propiedad de amnesia (pérdida de memoria).

Propiedad de Markov

El proceso adaptado $X = \{X(t)\}_{t\geq 0}$ es una Cadena de Markov a Tiempo Continuo si satisface la condición de Markov relativa a la filtración $\mathbb{F}$: para cualquier conjunto discreto de tiempos

$$0 \leq t_1 < t_2 < ... < t_n < t$$

y cualquier secuencia de estados

$$i_1, i_2, ..., i_n, i, j \in S,$$

se cumple que

$$\mathbb{P}(X(t) = j \mid X(t_1) = i_1, ..., X(t_n) = i, X(s) = i) = \mathbb{P}(X(t) = j \mid X(s) = i).$$

De manera equivalente y más elegante mediante la esperanza condicional:

$$\mathbb{P}(X(t) = j \mid \mathcal{F}_s) = \mathbb{P}(X(t) = j \mid X(s)) \quad \text{c.s. (casi seguramente)}$$

para todo $0 \leq s \leq t$.

Probabilidades de transición y homogeneidad

Definimos la probabilidad de transición del estado i al estado j en el intervalo $[s, t]$ como

$$p_{ij}(s, t) = \mathbb{P}(X(t) = j \mid X(s) = i).$$

Se dice que la CTMC es **homogénea en el tiempo** si $p_{ij}(s, t)$ depende únicamente de la longitud del intervalo temporal

$$\tau = t - s.$$

En tal caso, denotamos la probabilidad de transición como

$$p_{ij}(\tau) = \mathbb{P}(X(s + \tau) = j \mid X(s) = i).$$

Sean estas probabilidades las entradas de una matriz estocástica

$$P(\tau) = [p_{ij}(\tau)]_{i,j\in S}.$$

Ecuaciones de Chapman-Kolmogorov

Para cualquier $s, t \geq 0$, la familia de matrices de transición $\{P(t)\}_{t\geq 0}$ forma un semigrupo de operadores estocásticos, satisfaciendo

$$P(t+s) = P(t)P(s)$$

lo que equivale a

$$p_{ij}(t+s) = \sum_{k\in S} p_{ik}(t)p_{kj}(s).$$

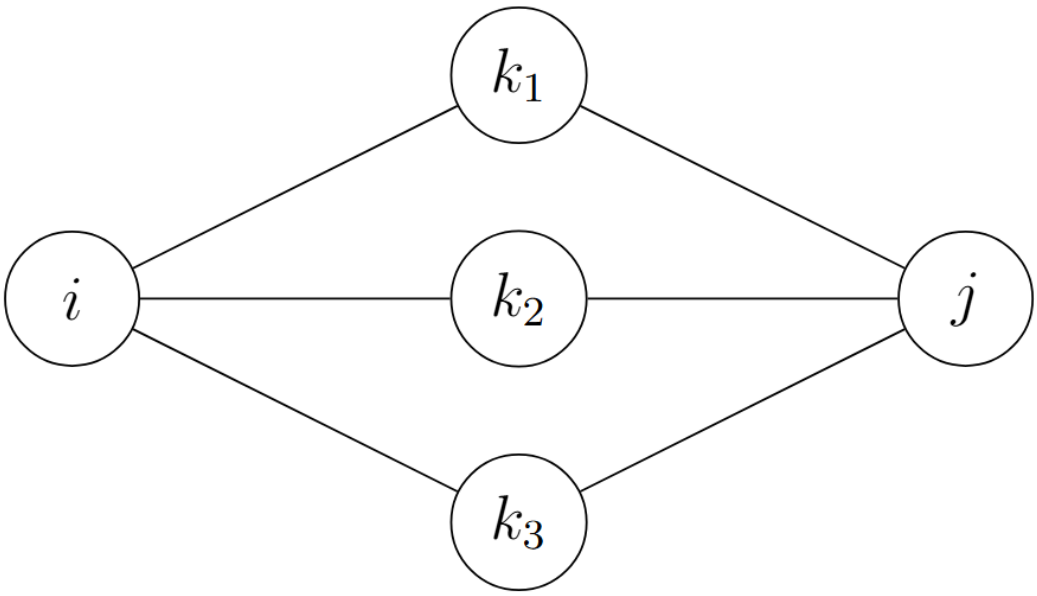


Figura 4.1: La probabilidad de transición $p_{ij}(t+s)$ se obtiene sumando las contribuciones de todos los posibles estados intermedios k.

Por otro lado, también se tiene la existencia de los HMM categóricos.

Modelos Ocultos de Markov Categóricos (HMM)

Un Modelo Oculto de Márkov Categórico (Categorical HMM, a menudo también llamado HMM Discreto o Multinomial) es una variante de los modelos HMM donde las observaciones emitidas por los estados ocultos son valores discretos y finitos que pertenecen a categorías específicas, en lugar de ser valores numéricos continuos.

En cualquier HMM, el sistema modela un proceso estocástico con estados que no son directamente visibles, y cada estado genera (emite) una observación que sí podemos medir.

4.2.1. Dinámica Infinitesimal y Generador de la Cadena

La evolución continua del color del pixel requiere una descripción mediante probabilidades de transición, lo cual nos lleva a la representación matricial de la derivada de $P(t)$.

Matriz generadora infinitesimal

Asumiendo que $p_{ij}(t)$ es diferenciable por la derecha en $t = 0$, definimos la probabilidad de transición q_{ij} del estado i al estado j (donde $i \neq j$) como:

$$q_{ij} = \lim_{h \to 0^+} \frac{p_{ij}(h)}{h} \geq 0.$$

Y para el caso $i = j$, definimos la probabilidad de salida del estado i como $v_i = -q_{ii}$, dada por

$$q_{ii} = \lim_{h \to 0^+} \frac{p_{ii}(h) - 1}{h} = -\sum_{j \neq i} q_{ij} \leq 0.$$

La matriz

$$Q = [q_{ij}]_{i,j \in S}$$

se denomina matriz generadora (o matriz Q) de la cadena. Sus filas suman cero:

$$\sum_{j \in S} q_{ij} = 0.$$

Ecuaciones diferenciales de Kolmogorov

Bajo condiciones de regularidad (que se cumplen trivialmente si el espacio de estados del color S es finito), la matriz de transición $P(t)$ es la solución única a las ecuaciones diferenciales matriciales de Kolmogorov Hacia Adelante (Forward) y Hacia Atrás (Backward) @Norris1998 :

- **Forward:**

$$\frac{d}{dt} P(t) = P(t) Q$$

- **Backward:**

$$\frac{d}{dt}P(t) = QP(t)$$

Cuya solución formal, bajo la condición inicial $P(0) = I$ (matriz identidad), es la matriz exponencial

$$P(t) = e^{Qt} = \sum_{n=0}^{\infty} \frac{(Qt)^n}{n!}.$$

4.3. Utilidad hacia Cadenas de Márkov Ocultas (HMM)

La fundamentación teórica precedente modela la dinámica subyacente determinista de la matriz de transición y la evolución estocástica de las trayectorias del sistema. Para extender esta arquitectura a un Modelo Oculto de Márkov (HMM) en el contexto de la inferencia y análisis de píxeles en imágenes que contienen letras del alfabeto latino, se formaliza la siguiente estructura bivariada:

Proceso latente e Inobservable

El proceso de Markov a tiempo continuo

$$X = \{X(t)\}_{t\geq 0}$$

sobre el espacio de estados S, caracterizado por su matriz generadora infinitesimal Q, constituye el *proceso latente*. Las alteraciones estocásticas inducidas por Q a lo largo del parámetro t (representando una métrica de degradación física o divergencia espacial) rigen las transiciones hacia estados degradados, manteniéndose inaccesibles a la medición directa.

La variable $X(t)$ describe el estado ideal o "verdadero" del pixel.

Proceso de observación empírica

Sea

$$(O, \mathcal{P}(O))$$

el espacio medible de las posibles mediciones obtenidas por instrumentos.

Se postula un *proceso de observación* acoplado

$$Y = \{Y(t)\}_{t\geq 0},$$

compuesto por variables aleatorias

$$Y(t) : \Omega \to O$$

que representan la lectura empírica o ruidosa en el parámetro t.

En este caso O denota el conjunto finito de colores o intensidades registrables por un sensor óptico y Y es la lectura ruidosa del pixel en el tiempo t.

Matriz de emisión e independencia condicional

La dependencia estocástica entre ambos procesos queda unívocamente determinada por la condición de independencia local.

Para cualquier instante $t \geq 0$, la observación $Y(t)$ es condicionalmente independiente de toda la historia previa del sistema dado el estado concurrente $X(t)$.

Formalmente, para cualquier $y \in O$ y $x \in S$:

$$\mathbb{P}(Y(t) = y \mid \mathcal{F}_t^X, \mathcal{F}_{t^-}^Y) = \mathbb{P}(Y(t) = y \mid X(t) = x) \quad \text{c.s.}$$

donde $\mathcal{F}_t^X$ y $\mathcal{F}_{t^-}^Y$ denotan las filtraciones **?@nte-espacio-filtrado** del historial latente y observable, respectivamente.

Estas probabilidades definen las entradas de la matriz de emisión estocástica

$$E = [e_{xy}],$$

donde

$$e_{xy} = \mathbb{P}(Y(t) = y \mid X(t) = x).$$

4.4. Inferencia y Filtrado en el Modelo Oculto

Una vez establecida la dinámica del sistema, es necesario desarrollar la maquinaria analítica para inferir el estado latente del pixel a partir de las observaciones empíricas.

Supongamos que disponemos de una secuencia de observaciones

$$Y = (y_1, y_2, \dots, y_K)$$

en instantes discretos de tiempo (o espacio)

$$0 \leq t_1 < t_2 < \dots < t_K \leq T.$$

Filtro hacia adelante y verosimilitud

Definimos la variable de avance $\alpha_k(i)$ como la probabilidad conjunta de observar la secuencia parcial hasta el k-ésimo instante y que el estado oculto en dicho instante sea $i \in S$:

$$\alpha_k(i) = \mathbb{P}(Y(t_1) = y_1, \dots, Y(t_k) = y_k, X(t_k) = i).$$

Esta cantidad se calcula recursivamente mediante la ecuación de Chapman-Kolmogorov **?@imp-chapman-kolmogorov** y la matriz de emisión E **?@nte-matriz-emision**:

$$\alpha_k(j) = \left(\sum_{i \in S} \alpha_{k-1}(i)\, p_{ij}(\Delta t_k) \right) \mathbb{P}(Y(t_k) = y_k \mid X(t_k) = j)$$

donde

$$\Delta t_k = t_k - t_{k-1}$$

y

$$p_{ij}(\Delta t_k)$$

es la entrada correspondiente de la matriz de transición

$$P(\Delta t_k) = e^{Q \Delta t_k}.$$

La verosimilitud total de la secuencia de observaciones es simplemente

$$\mathbb{P}(Y) = \sum_{i \in S} \alpha_K(i).$$

Decodificación óptima global

Para reconstruir la trayectoria más probable, buscamos la secuencia de estados latentes

$$\hat{X} = (\hat{x}_1, \ldots, \hat{x}_K)$$

que maximice la probabilidad condicional conjunta

$$\hat{X} = \arg \max_{x_1, \ldots, x_K} \mathbb{P}\left(X(t_1) = x_1, \ldots, X(t_K) = x_K \mid Y(t_1) = y_1, \ldots, Y(t_K) = y_K\right).$$

A diferencia de la función máx, que devuelve el **valor numérico** de la probabilidad más alta alcanzada, el operador arg máx (argumento del máximo) devuelve el **conjunto de entradas** que producen ese valor máximo. En este contexto, no nos interesa saber cuál es el porcentaje exacto de la probabilidad máxima, sino descubrir específicamente **cuál es la secuencia exacta de estados latentes** $\hat{X}$ que hace que esa probabilidad sea máxima

La solución sistemática a este problema de programación dinámica espacial/temporal se obtiene mediante la adaptación del Algoritmo de Viterbi @rabiner1989tutorial para cadenas a tiempo continuo, evaluando las transiciones máximas sobre

$$P(\Delta t_k).$$

Para los pixeles es la trayectoria más probable de los colores verdaderos del pixel (el problema de decodificación).

4.4.1. Ejemplo ilustrativo del algoritmo Forward–Backward

Para ilustrar el funcionamiento de los algoritmos Forward y Backward en el contexto del reconocimiento de imágenes, consideremos un píxel cuyo color verdadero puede encontrarse en uno de dos estados ocultos:

$$S = \{B, N\},$$

donde:

- B: píxel verdaderamente blanco.
- N: píxel verdaderamente negro.

Debido al ruido en el proceso de adquisición de la imagen, el color observado puede diferir del color real.

Supongamos que la dinámica del píxel está descrita por la matriz de transición

$$P = \begin{pmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{pmatrix},$$

donde, por ejemplo,

$$p_{BN} = 0.2$$

representa la probabilidad de que un píxel blanco pase al estado negro en el siguiente instante.

Asimismo, consideremos la siguiente matriz de emisión:

$$E = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix},$$

cuyas columnas corresponden a las observaciones

$$(O_B, O_N),$$

es decir, observar blanco u observar negro. Por ejemplo,

$$P(O_B \mid B) = 0.9, \qquad P(O_B \mid N) = 0.2.$$

Finalmente, asumimos una distribución inicial

$$\pi = (0.6, 0.4),$$

y que la secuencia observada es

$$Y = (O_B, O_N).$$

Esto significa que el sensor registra blanco en el primer instante y negro en el segundo.

4.4.2. Cálculo Forward

La variable Forward se define como

$$\alpha_k(i) = P(y_1, \dots, y_k, X_k = i).$$

Para la primera observación O_B:

$$\alpha_1(B) = P(X_1 = B)\, P(O_B \mid B) = 0.6(0.9) = 0.54,$$

$$\alpha_1(N) = P(X_1 = N)\, P(O_B \mid N) = 0.4(0.2) = 0.08.$$

Calculamos primero la probabilidad de alcanzar el estado blanco:

$$\sum_{i \in S} \alpha_1(i) p_{iB} = 0.54(0.8) + 0.08(0.3) = 0.456.$$

Multiplicando por la probabilidad de emisión correspondiente,

$$\alpha_2(B) = 0.456(0.1) = 0.0456.$$

Análogamente, para el estado negro:

$$\sum_{i \in S} \alpha_1(i) p_{iN} = 0.54(0.2) + 0.08(0.7) = 0.164,$$

y por tanto,

$$\alpha_2(N) = 0.164(0.8) = 0.1312.$$

Verosimilitud de la secuencia observada

La probabilidad total de observar la secuencia Y es

$$P(Y) = \alpha_2(B) + \alpha_2(N),$$

es decir,

$$P(Y) = 0.0456 + 0.1312 = 0.1768.$$

Cálculo Backward

La variable Backward se define mediante

$$\beta_k(i) = P(y_{k+1}, \dots, y_T \mid X_k = i).$$

Como la secuencia contiene únicamente dos observaciones,

$$\beta_2(B) = \beta_2(N) = 1.$$

Retrocediendo un paso:

$$\beta_1(B) = 0.8(0.1)(1) + 0.2(0.8)(1) = 0.24,$$

y

$$\beta_1(N) = 0.3(0.1)(1) + 0.7(0.8)(1) = 0.59.$$

Inferencia del estado oculto

Una de las principales ventajas del algoritmo Forward–Backward es que permite calcular la probabilidad posterior de cada estado oculto.

Para el estado blanco en el primer instante,

$$\gamma_1(B) = P(X_1 = B \mid Y) = \frac{\alpha_1(B)\beta_1(B)}{P(Y)},$$

por lo que

$$\gamma_1(B) = \frac{0.54(0.24)}{0.1768} = 0.733.$$

De forma análoga,

$$\gamma_1(N) = \frac{\alpha_1(N)\beta_1(N)}{P(Y)} = \frac{0.08(0.59)}{0.1768} = 0.267.$$

En consecuencia,

$$P(X_1 = B \mid Y) \approx 73.3\,\%,$$

mientras que

$$P(X_1 = N \mid Y) \approx 26.7\,\%.$$

Interpretación

A pesar de que el sensor observa un píxel blanco en el primer instante y negro en el segundo, el algoritmo Forward–Backward determina que existe aproximadamente un 73 % de probabilidad de que el color verdadero inicial haya sido blanco. Este resultado muestra cómo un Modelo Oculto de Markov combina la información proporcionada por las observaciones ruidosas con la dinámica de transición del sistema para inferir el estado real subyacente del píxel.

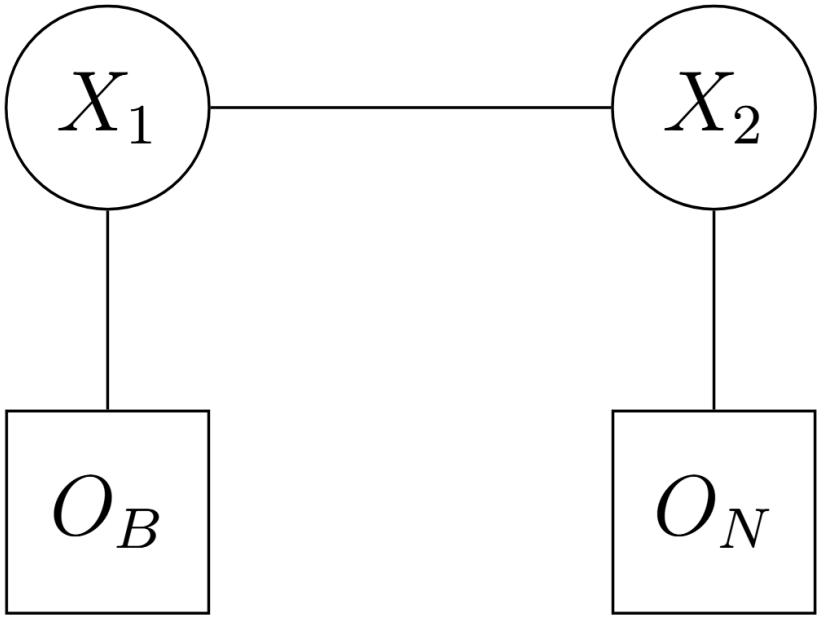


Figura 4.2: Modelo Oculto de Markov para un píxel con dos estados ocultos y dos observaciones.

4.5. Estimación de Parámetros y Calibración del Modelo

En la práctica, la matriz generadora infinitesimal Q (que dicta la probabilidad de degradación del color para el caso de estudio) y las probabilidades de emisión empírica rara vez son conocidas a priori y deben ser estimadas a partir de un dataset de imágenes (el alfabeto latino modificado).

Algoritmo de Expectation-Maximization (Baum-Welch) La estimación por máxima verosimilitud del conjunto de parámetros del modelo

$$\Theta = \{Q, E\}$$

dado un conjunto de secuencias observadas se realiza mediante un proceso iterativo.

- **Paso E (Expectation):** Se calculan las estadísticas suficientes esperadas del sistema latente (tiempo esperado de permanencia en cada estado de color y número esperado de transiciones) condicionado a las observaciones y a la estimación actual de los parámetros en el paso m $\Theta^{(m)}$, utilizando las variables recursivas Forward y Backward.
- **Paso M (Maximization):** Se actualizan los parámetros $\Theta^{(m+1)}$ para maximizar la función Q de verosimilitud esperada. En particular, la estimación de las probabilidades de transición empíricas se formula restringiendo las soluciones para garantizar que las filas de $Q^{(m+1)}$ sigan sumando cero y $q_{ij} \geq 0$ para $i \neq j$. :::

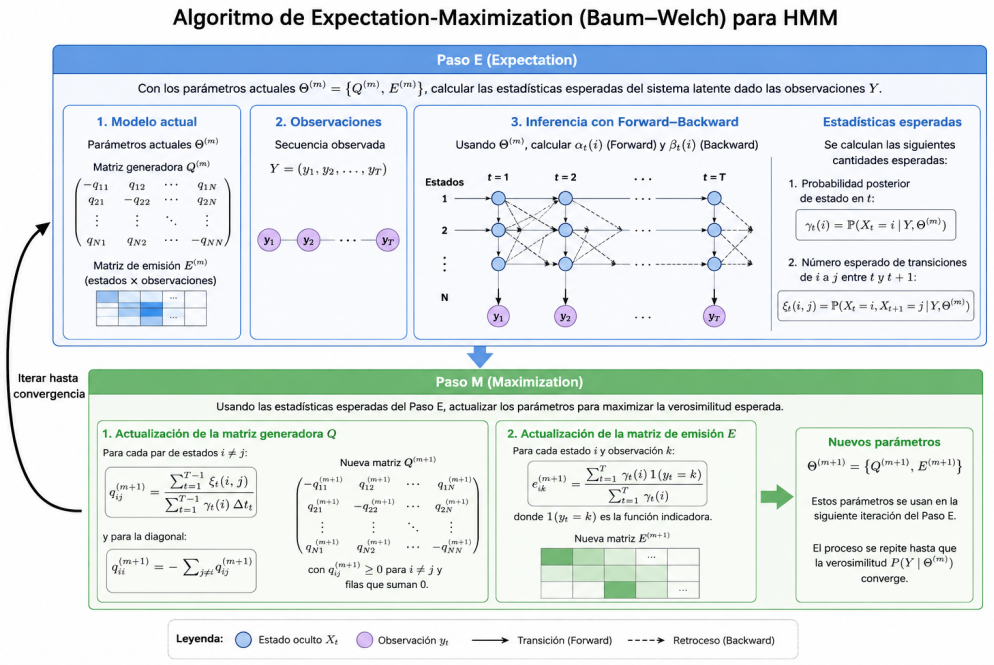


Figura 4.3: Esquema general del algoritmo Expectation–Maximization (Baum–Welch) para un Modelo Oculto de Markov. En el Paso E se estiman las probabilidades posteriores de los estados ocultos y las transiciones esperadas mediante los algoritmos Forward–Backward. En el Paso M se actualizan iterativamente la matriz generadora y la matriz de emisión para maximizar la verosimilitud de las observaciones.

5 Fundamentos matemáticos de las Redes Neuronales Convolucionales

5.1. Fundamentos CNN

En la tarea de clasificación de imágenes, cada glifo es representado mediante una estructura matricial que codifica su información visual.

Los glifos presentan variaciones en forma, trazo y estilo, lo que dificulta su identificación mediante métodos tradicionales. En este contexto, las CNN resultan particularmente adecuadas debido a su capacidad para extraer características espaciales relevantes directamente a partir de los datos, tales como bordes, curvas y patrones geométricos distintivos.

5.2. Formulación Matemática de una CNN

Una red neuronal convolucional puede modelarse como una aplicación parametrizada:

$$f(\cdot, \theta) : \mathcal{X} \subseteq \mathbb{R}^{H \times W \times C} \longrightarrow \mathbb{R}^k \tag{5.1}$$

donde:

- $\mathcal{X}$ es el espacio de entrada (por ejemplo, imágenes),
- H representa la altura (height) de la imagen, es decir, el número de filas,
- W representa el ancho (width), es decir, el número de columnas,
- C representa el número de canales (channels), que corresponde a la cantidad de valores asociados a cada píxel,
- k es la dimensión del espacio de salida (por ejemplo, número de clases en clasificación),
- θ denota el conjunto total de parámetros entrenables del modelo.

5.3. Representación Tensorial de Imágenes

Una imagen digital se representa como un tensor de orden tres:

$$X \in \mathbb{R}^{H \times W \times C} \tag{5.2}$$

Esto implica que para cada posición espacial (i, j):

$$X(i,j) = (x_1, x_2, \dots, x_C) \in \mathbb{R}^C \tag{5.3}$$

Por tanto, cada píxel contiene un vector que describe información en distintos canales.

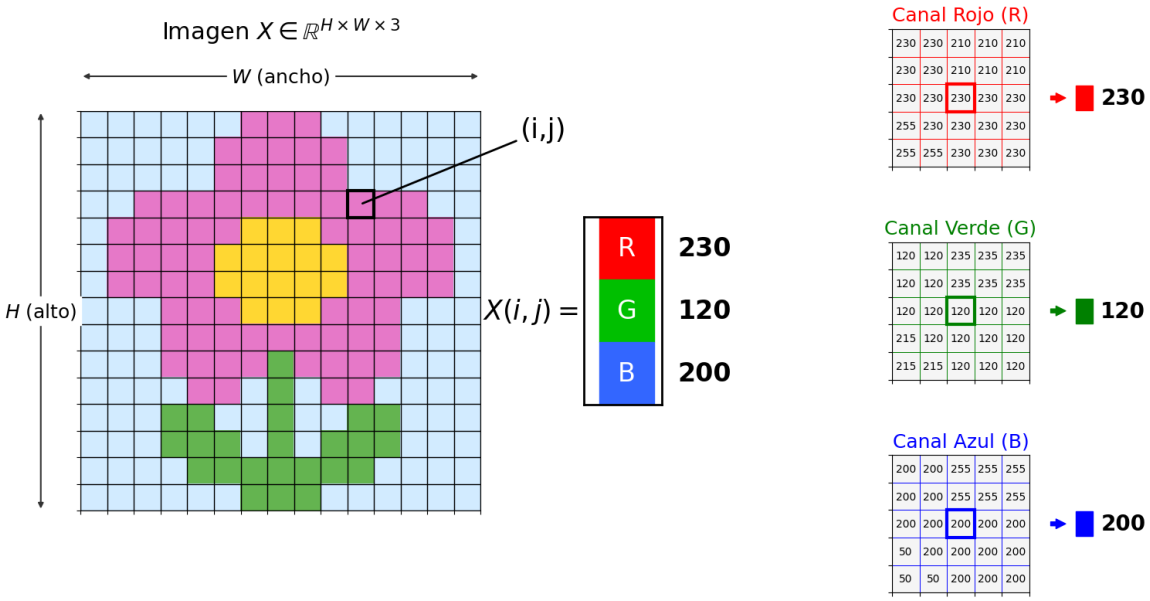


Figura 5.1: En la imágen se observa que cada píxel está formado en realidad por la combinación de valores de los tres canales.

5.3.1. Estructura Funcional de la Red

Una CNN se construye como la composición de funciones:

$$f(x;\theta) = f^{(L)} \circ f^{(L-1)} \circ \cdots \circ f^{(1)}(x) \tag{5.4}$$

donde $L \in \mathbb{N}$ es el número total de capas de la red y cada $f^{(l)}$ representa una transformación asociada a la capa l-ésima.

Asimismo, los parámetros globales se expresan como:

$$\theta = \left\{\theta^{(1)}, \theta^{(2)}, \dots, \theta^{(L)}\right\} \tag{5.5}$$

5.3.2. Parámetros del Modelo

Los parámetros de una CNN están constituidos principalmente por:

- **Kernels** (filtros convolucionales),

- **Sesgos** asociados a cada filtro.

Estos parámetros determinan completamente la función $f(x;\theta)$ y son ajustados mediante procedimientos de optimización basados en gradiente.

5.3.3. Kernels Convolucionales

Definición Formal

Un kernel es un tensor de parámetros que define un operador lineal local.

Caso unicanal:

$$K \in \mathbb{R}^{k_h \times k_w} \tag{5.6}$$

cuyos elementos se indexan como:

$$K(u,v), \quad u \in \{0,\dots,k_h-1\},\ v \in \{0,\dots,k_w-1\}$$

Caso multicanal:

$$K \in \mathbb{R}^{k_h \times k_w \times C} \tag{5.7}$$

con entradas:

$$K(u,v,c), \quad u \in \{0,\dots,k_h-1\},\ v \in \{0,\dots,k_w-1\},\ c \in \{1,\dots,C\}$$

Interpretación Geométrica

Sea $X_{(i,j)}$ una subregión de la imagen obtenida al centrar el kernel en la posición (i,j), cuyos elementos están dados por:

$$X(i+u, j+v, c)$$

para los mismos índices (u,v,c) definidos anteriormente.

El producto interno:

$$\langle K, X_{(i,j)} \rangle \tag{5.8}$$

mide la similitud entre el patrón definido por el kernel y la región local de la imagen.

Compartición de parámetros

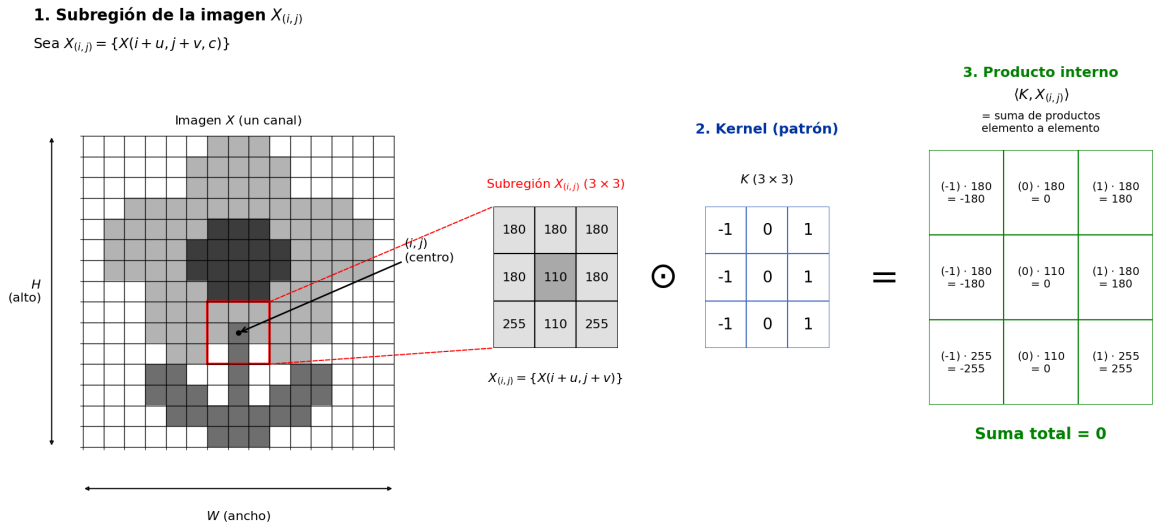


Figura 5.2: Se observa un ejemplo de como funciona un kernel especial para detectar bordes al aplicarlo a una subregión.

Una propiedad fundamental de las redes neuronales convolucionales es la **compartición de parámetros**, la cual establece que los coeficientes del kernel no dependen de la posición espacial en la que se aplica la operación.

Formalmente, si K es un kernel convolucional, entonces:

$$K(u, v, c) \text{ es independiente de la posición } (i, j)$$

Esto implica que el mismo kernel es utilizado para procesar todas las regiones locales de la entrada.

Reducción del número de parámetros

En ausencia de compartición de parámetros, cada posición (i, j) requeriría un conjunto distinto de pesos, lo cual conduciría a un número total de parámetros del orden:

$$H \cdot W \cdot k_h \cdot k_w \cdot C$$

Sin embargo, al emplear un único kernel compartido, el número de parámetros se reduce a:

$$k_h \cdot k_w \cdot C$$

lo cual representa una reducción significativa en la complejidad del modelo.

Invarianza traslacional

La compartición de parámetros induce una propiedad de **equivarianza por traslación**. Sea $\tau_{a,b}$ un operador de traslación definido sobre la entrada. Entonces, la operación de convolución satisface:

$$\mathcal{T}_K(\tau_{a,b}X) = \tau_{a,b}(\mathcal{T}_K X) \tag{5.9}$$

Esto implica que si una característica aparece en diferentes posiciones espaciales, el modelo será capaz de detectarla de manera consistente.

En consecuencia, la red no depende de la posición absoluta de los patrones, sino únicamente de su estructura local.

5.4. Operación de Convolución

La operación fundamental en una red neuronal convolucional es la denominada **convolución discreta**. No obstante, es importante señalar que en el contexto de aprendizaje profundo, la operación implementada corresponde en realidad a la **correlación cruzada**, ya que no se realiza la inversión del kernel. A pesar de esta diferencia técnica, en la literatura se mantiene el término "convolución'' por convención.

Esta operación define un mecanismo mediante el cual se evalúa la similitud entre un patrón local (kernel) y distintas regiones de la imagen de entrada.

5.4.1. Caso Unicanal

Sea una imagen unicanal representada por:

$$X \in \mathbb{R}^{H \times W}$$

y un kernel:

$$K \in \mathbb{R}^{k_h \times k_w}$$

La operación de convolución produce una salida Y, definida en cada posición (i, j) como:

$$Y(i,j) = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u,v)\, X(i+u, j+v) \tag{5.10}$$

donde:

- (i, j) denota la posición de la ventana deslizante,
- (u, v) recorre las coordenadas internas del kernel,
- $X(i + u, j + v)$ representa la región local de la imagen.

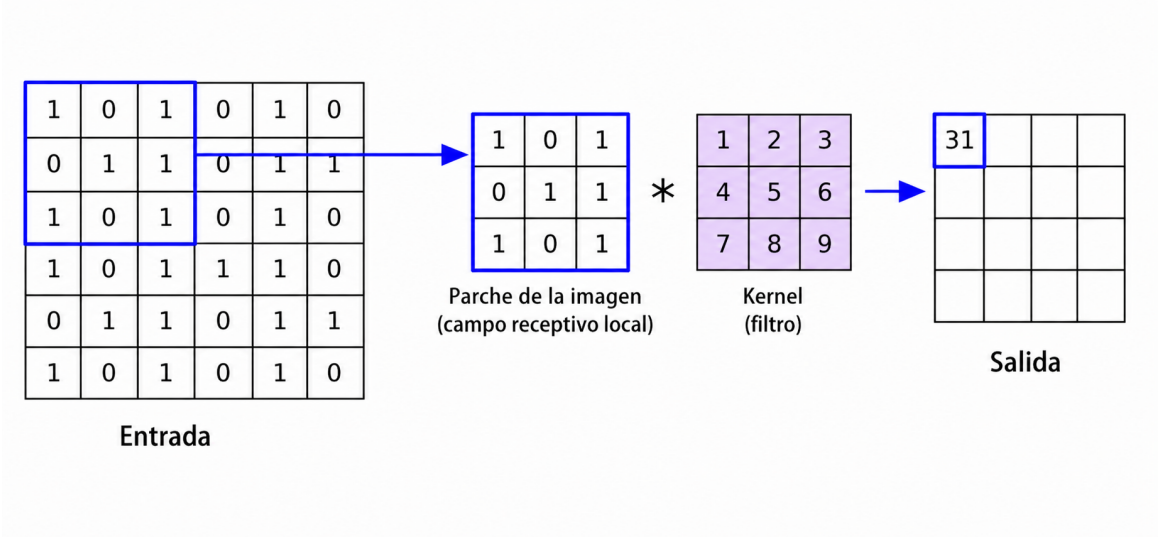


Figura 5.3: En la imágen se observa como opera un kernel. Se visualiza los valores correspondientes a la imágen original compuesta de ceros y unos, se selecciona una subregión o parche y usando el filtro dado se realizan las operaciones correspondientes de multiplicación y suma, de modo que el resultado es registrado en el mapa de salidas, que constituye una nueva imágen más pequeña que la original.

Interpretación como ventana deslizante

La ecuación anterior describe un proceso en el cual el kernel se desplaza sistemáticamente sobre la imagen. En cada posición, se selecciona una submatriz de X de dimensiones $k_h \times k_w$, sobre la cual se realiza una operación de agregación ponderada.

Formalmente, definimos la submatriz local como:

$$X_{(i,j)} = \{X(i+u, j+v)\}_{u,v}$$

Por lo tanto, la salida en (i, j) depende únicamente de una vecindad local de la imagen, lo cual establece la propiedad de **conectividad local**.

Interpretación como producto interno

La expresión de la convolución puede reescribirse como un producto interno en el espacio vectorial $\mathbb{R}^{k_h \times k_w}$:

$$Y(i,j) = \langle K, X_{(i,j)} \rangle \tag{5.11}$$

donde el producto interno se define como:

$$\langle A, B \rangle = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} A(u,v)\, B(u,v)$$

Esta formulación pone de manifiesto que la convolución mide el grado de alineación entre el kernel y la región local.

Interpretación geométrica

Desde un punto de vista geométrico, el valor $Y(i,j)$ representa la proyección del vector $X_{(i,j)}$ sobre el vector K en el espacio euclidiano de dimensión $k_h k_w$. En consecuencia:

- Si $X_{(i,j)}$ es similar a K, entonces $Y(i,j)$ será grande.
- Si son ortogonales, el resultado será cercano a cero.
- Si son opuestos, el valor será negativo.

Esto permite interpretar al kernel como un **detector de patrones**.

Naturaleza lineal de la operación

La convolución es un operador lineal. Es decir, para cualesquiera tensores X_1, X_2 y escalares α, β, se cumple:

$$\mathcal{T}_K(\alpha X_1 + \beta X_2) = \alpha \mathcal{T}_K(X_1) + \beta \mathcal{T}_K(X_2) \tag{5.12}$$

donde $\mathcal{T}_K$ denota el operador de convolución asociado al kernel K.

Relación con la correlación cruzada

Es importante destacar que la definición utilizada en redes neuronales corresponde a la correlación cruzada:

$$Y(i,j) = \sum_{u,v} K(u,v)\, X(i+u, j+v)$$

mientras que la convolución clásica implicaría una inversión del kernel:

$$Y(i,j) = \sum_{u,v} K(u,v)\, X(i-u, j-v)$$

Sin embargo, dado que los parámetros del kernel son aprendidos durante el entrenamiento, ambas formulaciones resultan equivalentes en términos de capacidad representacional.

Dependencia local

Finalmente, se observa que:

$$Y(i,j) \text{ depende únicamente de } X_{(i,j)}$$

lo cual implica que cada salida está influenciada únicamente por una región local de la entrada. Esta propiedad es clave para la eficiencia computacional y la capacidad de generalización del modelo.

5.4.2. Convolución Multicanal

En el caso general, las imágenes no son unicanal, sino que poseen múltiples canales (por ejemplo, RGB). En consecuencia, la operación de convolución debe extenderse para considerar simultáneamente la información presente en cada canal.

Sea:

$$X \in \mathbb{R}^{H \times W \times C}$$

una imagen con C canales, y sea un kernel:

$$K \in \mathbb{R}^{k_h \times k_w \times C}$$

donde cada canal del kernel corresponde a un canal de la entrada.

La salida de la convolución en la posición (i, j) está dada por:

$$Y(i,j) = \sum_{c=1}^{C} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u,v,c)\, X(i+u, j+v, c) \tag{5.13}$$

Esta expresión indica que la contribución de cada canal se calcula de manera independiente y posteriormente se agregan (suman) todas las contribuciones.

La operación anterior puede interpretarse como la suma de convoluciones unicanal:

$$Y(i,j) = \sum_{c=1}^{C} Y^{(c)}(i,j)$$

donde:

$$Y^{(c)}(i,j) = \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K(u,v,c)\, X(i+u, j+v, c)$$

Cada término $Y^{(c)}(i,j)$ representa la contribución del canal c al valor final de la salida.

Interpretación como producto interno en espacio tensorial

Sea $X_{(i,j)}$ la subregión local de la imagen centrada en (i, j). Entonces:

$$X_{(i,j)} \in \mathbb{R}^{k_h \times k_w \times C}$$

Al vectorizar ambos tensores:

$$\text{vec}(X_{(i,j)}) \in \mathbb{R}^{k_h k_w C}, \quad \text{vec}(K) \in \mathbb{R}^{k_h k_w C}$$

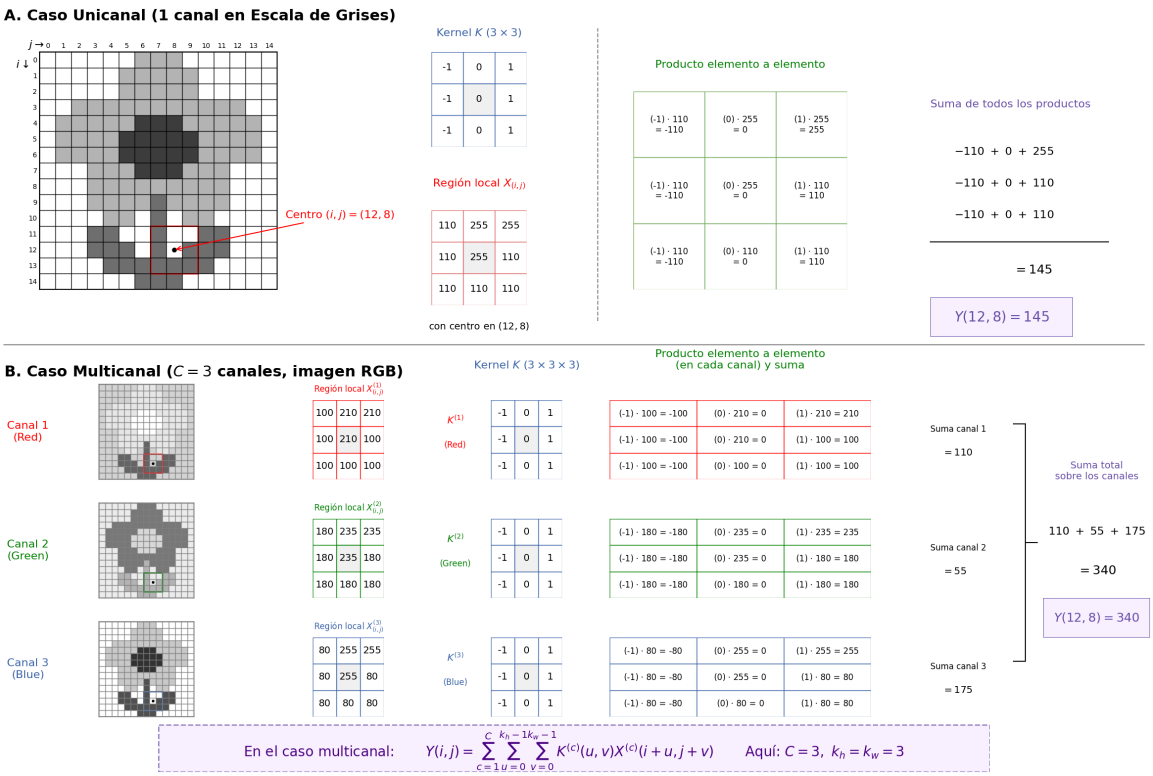


Figura 5.4: Se representa la manera en la que opera un kernel dependiendo de si se trata de una imágen a blanco y negro (unicanal) o una a color en RGB (multicanal), ambas tomando la misma subregión correspondiente.

la convolución puede escribirse como:

$$Y(i,j) = \text{vec}(K)^\top \text{vec}(X_{(i,j)}) \tag{5.14}$$

Esto muestra que la operación es un producto interno en un espacio euclidiano de dimensión $k_h k_w C$.

Interpretación geométrica

Desde un punto de vista geométrico, cada región local de la imagen se interpreta como un punto en el espacio $\mathbb{R}^{k_h k_w C}$, y el kernel define una dirección en dicho espacio.

Por tanto:

- valores grandes de $Y(i,j)$ indican alta alineación entre el patrón y la región,
- valores cercanos a cero indican baja correlación,
- valores negativos indican oposición estructural.

En consecuencia, el kernel actúa como un detector de patrones multicanal.

Naturaleza lineal

La convolución multicanal es un operador lineal. Es decir, para cualesquiera tensores X_1, X_2 y escalares α, β, se cumple:

$$\mathcal{T}_K(\alpha X_1 + \beta X_2) = \alpha \mathcal{T}_K(X_1) + \beta \mathcal{T}_K(X_2)$$

donde $\mathcal{T}_K$ denota el operador de convolución asociado al kernel K.

Interpretación estructural

Es importante destacar que, a diferencia del caso unicanal, el kernel multicanal no detecta únicamente patrones espaciales, sino también **correlaciones entre canales**.

Por ejemplo, en imágenes RGB, el kernel puede aprender a identificar combinaciones específicas de intensidades en los canales rojo, verde y azul, lo cual permite detectar patrones más complejos que aquellos basados únicamente en estructura espacial.

Forma Vectorizada

$$Y(i,j) = K^\top X_{(i,j)}$$

Múltiples Filtros

En una red neuronal convolucional, no se utiliza un único kernel, sino un conjunto de filtros que permiten extraer distintas características de la entrada. Este conjunto se denota como:

$$\left\{K^{(f)}\right\}_{f=1}^{F}, \quad K^{(f)} \in \mathbb{R}^{k_h \times k_w \times C} \tag{5.15}$$

donde F representa el número total de filtros.

Definición de la salida

Cada filtro $K^{(f)}$ actúa de manera independiente sobre la entrada y produce un mapa de activación asociado. Para cada posición (i, j), la salida correspondiente al filtro f está dada por:

$$Y_f(i,j) = \langle K^{(f)}, X_{(i,j)} \rangle \tag{5.16}$$

Por lo tanto, a cada filtro le corresponde una función:

$$Y_f : \{1, \dots, H'\} \times \{1, \dots, W'\} \to \mathbb{R}$$

Estructura tensorial de la salida

Al considerar todos los filtros simultáneamente, la salida completa se organiza como un tensor de orden tres:

$$Y \in \mathbb{R}^{H' \times W' \times F} \tag{5.17}$$

donde:

- H' y W' son las dimensiones espaciales de la salida,
- F es el número de canales de salida, también llamados **mapas de características**.

En particular, la componente $Y(:, :, f)$ corresponde al mapa de activación generado por el filtro $K^{(f)}$.

Interpretación funcional

Cada filtro puede interpretarse como un detector especializado en cierto tipo de patrón. En consecuencia, la colección de filtros define una familia de funciones:

$$\{\mathcal{T}_{K^{(f)}}\}_{f=1}^{F}$$

donde cada operador $\mathcal{T}_{K^{(f)}}$ extrae una característica distinta de la entrada.

De este modo, la salida de la capa puede interpretarse como la aplicación conjunta de múltiples detectores, produciendo una representación enriquecida de la imagen.

Interpretación como vector de características local

Para cada posición (i, j), la salida puede agruparse como un vector:

$$Y(i,j) = (Y_1(i,j), Y_2(i,j), \dots, Y_F(i,j)) \in \mathbb{R}^F \tag{5.18}$$

Esto implica que cada posición espacial de la imagen original es transformada en un vector de características de dimensión F.

Por tanto, la capa convolucional realiza una transformación:

$$\mathbb{R}^{k_h \times k_w \times C} \longrightarrow \mathbb{R}^F$$

aplicada localmente en cada vecindad de la imagen.

Interpretación geométrica

Desde una perspectiva geométrica, cada filtro $K^{(f)}$ define una dirección en el espacio $\mathbb{R}^{k_h k_w C}$. En consecuencia, el vector $Y(i,j)$ contiene las proyecciones de la región local $X_{(i,j)}$ sobre cada una de estas direcciones.

Esto permite interpretar la salida como una representación de la información local en una base (no necesariamente ortogonal) definida por los filtros aprendidos.

###Ejemplos de filtros

5.4.3. Observación estructural

Es importante notar que el número de filtros F determina la capacidad de representación de la capa. Un mayor número de filtros permite capturar una mayor variedad de patrones, pero también incrementa el número de parámetros y el costo computacional.

En capas profundas, los filtros tienden a representar características de mayor nivel semántico, construidas a partir de combinaciones de patrones detectados en capas anteriores.

5.5. Stride y Padding

5.5.1. Stride

El *stride* s controla el desplazamiento del kernel durante la operación de convolución. Es decir, determina cuántas posiciones se desplaza el kernel en cada paso.

Las dimensiones de salida están dadas por:

$$H' = \left\lfloor \frac{H - k_h + 2p}{s} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W - k_w + 2p}{s} \right\rfloor + 1 \tag{5.19}$$

donde:

- H, W son las dimensiones de la entrada,
- k_h, k_w son las dimensiones del kernel,
- p es el padding,

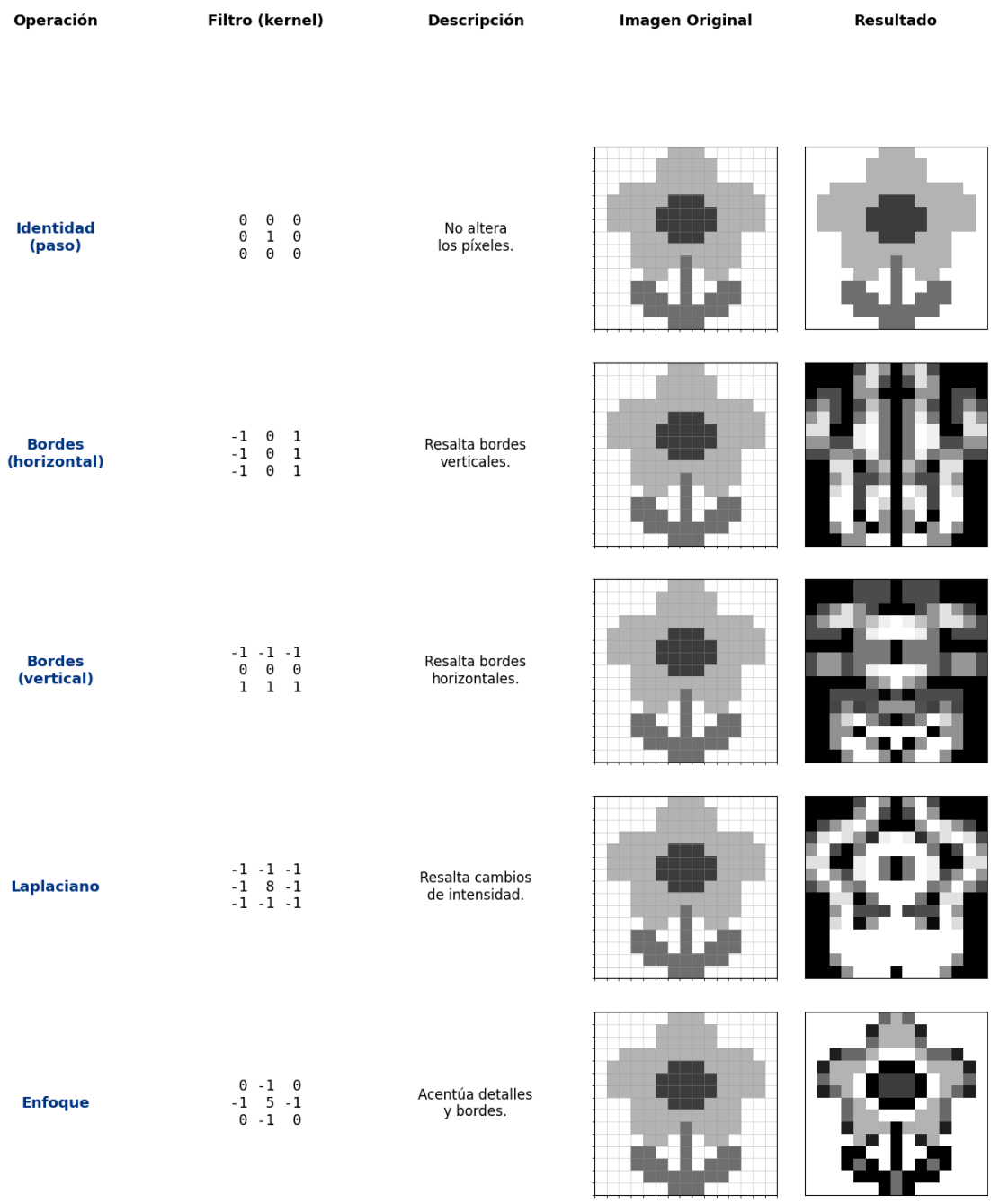


Figura 5.5: Los distintos filtros convolucionales permiten detectar características específicas en la imagen de entrada. En la imagen se muestran ejemplos representativos del efecto de diferentes kernels sobre una misma imagen.

- s es el stride.

Un valor mayor de s produce una reducción en la resolución espacial de la salida.

5.5.2. Padding

El *padding* es una operación que consiste en añadir valores (generalmente ceros) alrededor de la entrada antes de aplicar la convolución. Esto se hace para controlar el tamaño de la salida y preservar información en los bordes.

$$X_{\text{pad}} \in \mathbb{R}^{(H+2p)\times(W+2p)\times C}$$

El padding puede interpretarse como una extensión de la función de entrada X fuera de su dominio original. Formalmente, se define la extensión $\tilde{X}$ como:

$$\tilde{X}(i,j) = \begin{cases} X(i,j), & \text{si } (i,j) \text{ pertenece al dominio original,} \\ 0, & \text{en otro caso} \end{cases}$$

Esta extensión corresponde a una extensión por cero del dominio de la función.

En una capa convolucional, el padding se aplica antes de la convolución. Por lo tanto, la operación completa de una capa convolucional con padding se puede expresar como:

$$Y = \sigma((\text{pad}(X)) \star K + b)$$

donde:

- X es el tensor de entrada,
- $\text{pad}(X)$ es la entrada extendida mediante padding,
- $\star$ denota la operación de correlación cruzada,
- K es el kernel o filtro,
- b es el sesgo,
- σ es la función de activación.

Ejemplo ilustrativo

Para ilustrar el efecto del padding sobre las dimensiones de salida, se presenta el siguiente ejemplo:

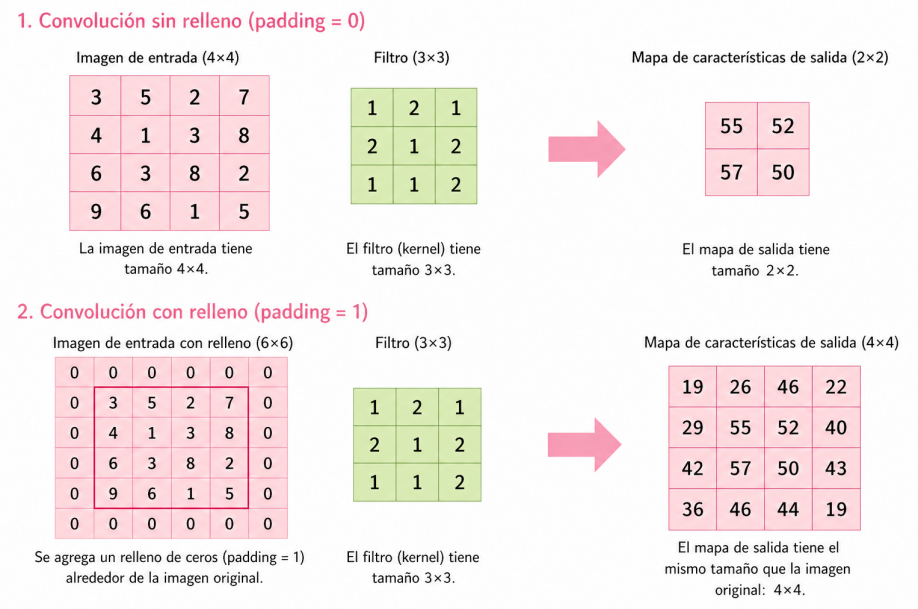


Figura 5.6: Ejemplo de convolución con y sin padding. La incorporación de padding permite controlar el tamaño de la salida. En particular, cuando se utiliza padding adecuado, es posible preservar las dimensiones espaciales de la imagen original tras la aplicación de la convolución. El color rosa identifica la imagen de entrada y la salida de la convolución, mientras que el verde representa el filtro empleado. En el caso del ejemplo con padding, las celdas correspondientes al relleno se distinguen al estar por fuera del cuadro rojo, para resaltar que no forman parte de la imagen original.

5.5.3. Función de activación

Una función de activación es una aplicación no lineal que se introduce después de una transformación afín, con el objetivo de dotar al modelo de la capacidad de aproximar funciones no lineales.

Definición

Sea $x \in \mathbb{R}^n$ un vector de entrada, $w \in \mathbb{R}^n$ un vector de pesos y $b \in \mathbb{R}$ un sesgo. Se define la preactivación como:

$$z = w^T x + b \tag{5.20}$$

La salida de la neurona está dada por:

$$y = \sigma(z) \tag{5.21}$$

donde $\sigma : \mathbb{R} \to \mathbb{R}$ es la función de activación.

No linealidad

Considérese una red neuronal profunda compuesta únicamente por transformaciones afines. En tal caso, la composición de dichas transformaciones satisface:

$$f(x) = W_L W_{L-1} \cdots W_1 x + b'$$

lo cual sigue siendo una transformación afín. Por consiguiente, sin la introducción de funciones de activación no lineales, la red carece de capacidad expresiva para modelar relaciones no lineales.

En una red neuronal convolucional (CNN), la salida de una capa se expresa como:

$$Y = \sigma(X \star K + b) \tag{5.22}$$

donde:

- X es el tensor de entrada
- $\star$ denota la operación de correlación cruzada
- K es el conjunto de filtros (kernels)
- b es el sesgo
- σ se aplica elemento a elemento

Ejemplos de funciones de activación

Imagenes_tesis/img_funciones_activacion.png

Algunas de las funciones de activación más utilizadas son las siguientes:

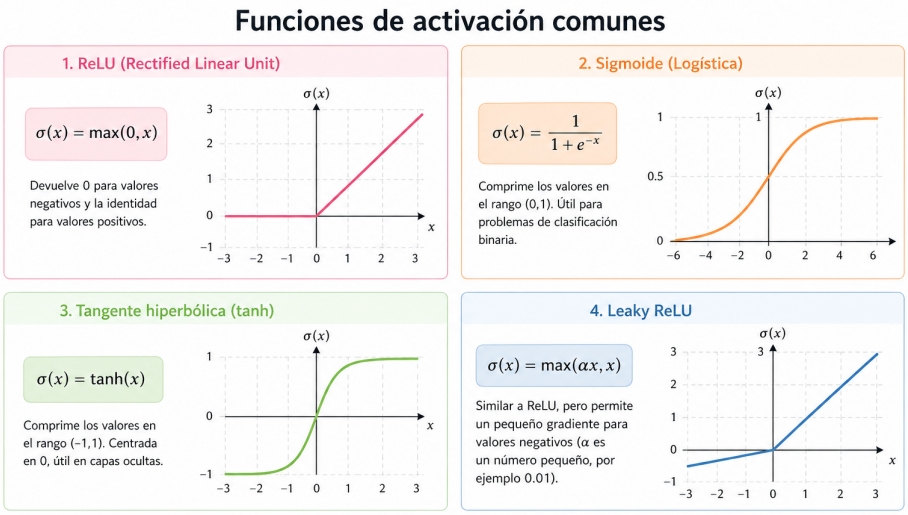


Figura 5.7: Gráficas y formulaciones de funciones de activación comunes (ReLU, Sigmoide, Tanh, Leaky ReLU).

ReLU (Rectified Linear Unit)

ReLU es eficientemente y se usa ampliamente en redes profundas.

$$\sigma(x) = \text{máx}(0, x) \tag{5.23}$$

Sigmoide

Sigmoide es útil en la capa de salida para clasificación binaria.

$$\sigma(x) = \frac{1}{1+e^{-x}} \tag{5.24}$$

Tangente hiperbólica

Tanh es útil cuando se necesita una salida centrada en 0.

$$\sigma(x) = \tanh(x) \tag{5.25}$$

Leaky ReLU

$$\sigma(x) = \text{máx}(0, x)$$

Propiedades fundamentales

Las funciones de activación empleadas en redes neuronales satisfacen típicamente:

- No linealidad, necesaria para incrementar la capacidad de representación del modelo
- Aplicación puntual (element-wise), preservando la estructura espacial en CNN
- Diferenciabilidad (al menos casi en todas partes), requisito esencial para métodos de optimización basados en gradiente

Formulación funcional

Desde una perspectiva funcional, una red neuronal profunda puede interpretarse como la composición de aplicaciones de la forma:

$$f_\theta = \sigma_L \circ T_L \circ \cdots \circ \sigma_1 \circ T_1 \tag{5.26}$$

donde cada transformación afín está dada por:

$$T_i(x) = W_i x + b_i$$

5.5.4. Pooling

El *pooling* es una operación que reduce la dimensión espacial de la representación, conservando las características más relevantes. Esta operación se aplica de manera independiente en cada canal del tensor de entrada.

Sea:

$$X \in \mathbb{R}^{H \times W \times C}$$

la entrada de la capa. El pooling produce una salida:

$$Y \in \mathbb{R}^{H' \times W' \times C}$$

donde las dimensiones H' y W' son menores que H y W.

Max Pooling

El *max pooling* es la forma más común de pooling. Para cada región local $X_{(i,j)}$, se define:

$$Y(i,j,c) = \max_{(u,v) \in \mathcal{R}} X(i+u, j+v, c) \tag{5.27}$$

donde $\mathcal{R}$ representa la ventana de pooling (por ejemplo, de tamaño 2×2).

Average Pooling

Otra variante es el *average pooling*, definido como:

$$Y(i,j,c) = \frac{1}{|\mathcal{R}|} \sum_{(u,v)\in\mathcal{R}} X(i+u, j+v, c) \tag{5.28}$$

El pooling sirve para reducir las dimensiones espaciales (ancho y alto) de un tensor, asegurando que se conserve la información más relevante. Aplicar esta técnica brinda grandes beneficios a los modelos: disminuye significativamente el costo computacional, introduce invariancia a pequeñas traslaciones en la imagen y ayuda a prevenir el sobreajuste. Además, es importante destacar que esta reducción se aplica de manera independiente a cada canal de profundidad del tensor

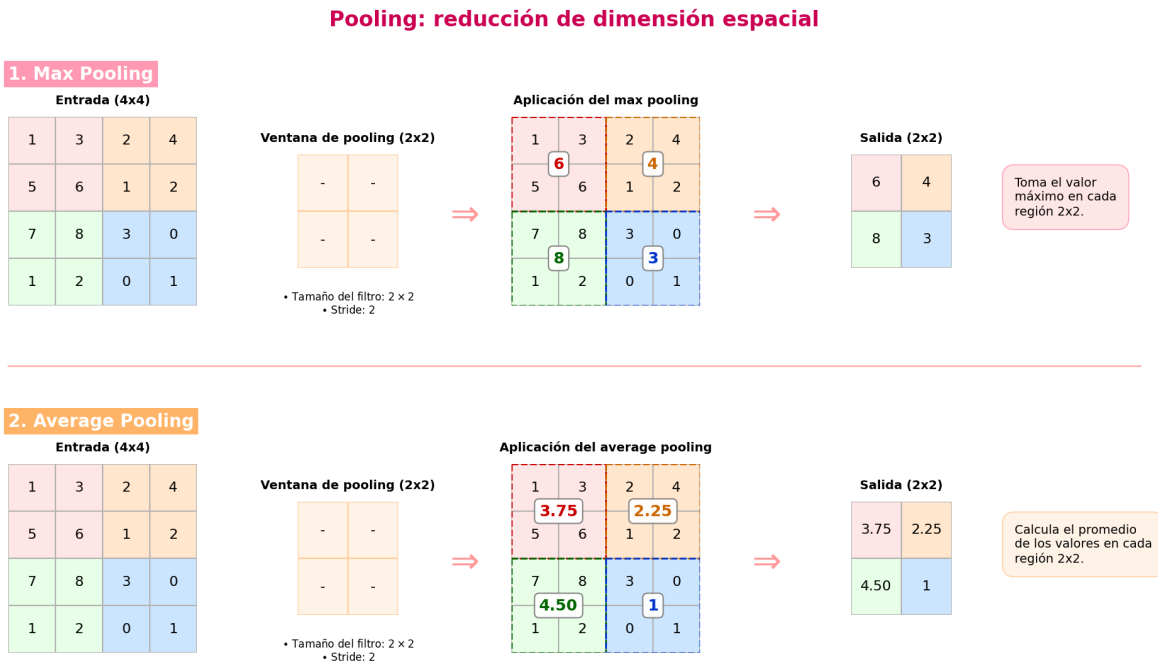


Figura 5.8: En la parte superior de la figura se ilustra el funcionamiento del Max Pooling utilizando una ventana de tamaño 2×2 y un stride de 2. La imagen de entrada se divide en cuatro regiones no superpuestas, identificadas mediante diferentes colores. Para cada región, el operador selecciona únicamente el valor máximo: en la región superior izquierda se obtiene 6, en la superior derecha 4, en la inferior izquierda 8 y en la inferior derecha 3. Estos valores conforman el mapa de salida de tamaño 2×2, preservando las características más destacadas de cada zona mientras se reduce la dimensionalidad espacial. De manera similar, en la parte inferior de la figura se muestra el proceso de Average Pooling con la misma ventana 2×2 y el mismo stride. En lugar de conservar el valor máximo, se calcula el promedio de los elementos contenidos en cada región. Así, las cuatro ventanas generan los valores 3.75, 2.25, 4.50 y 1, que constituyen la salida reducida. Este método produce una representación más suavizada de la información, manteniendo la tendencia general de los datos presentes en cada región de la imagen.

Propiedades

El pooling presenta las siguientes propiedades:

- Reduce la dimensionalidad espacial, disminuyendo el costo computacional
- Introduce una cierta invariancia a traslaciones pequeñas
- Opera de forma independiente en cada canal

Interpretación

El pooling puede interpretarse como una forma de resumen local de la información. En el caso del max pooling, se selecciona la característica más prominente dentro de cada región, mientras que el average pooling produce una media de las intensidades.

En arquitecturas profundas, el pooling contribuye a la construcción de representaciones más abstractas, al reducir progresivamente la resolución espacial.

5.5.5. Capas completamente conectadas

Las capas completamente conectadas (*Fully Connected*, FC) constituyen la etapa final de una red neuronal convolucional. Su función es combinar las características extraídas por las capas anteriores para realizar la tarea de clasificación o regresión.

Vectorización de la entrada

Sea:

$$X \in \mathbb{R}^{H \times W \times C}$$

la salida de la última capa convolucional. Esta se transforma en un vector mediante una operación de vectorización:

$$x = \mathrm{vec}(X) \in \mathbb{R}^{HWC} \tag{5.29}$$

Transformación afín

La capa completamente conectada aplica una transformación lineal seguida de un sesgo:

$$z = Wx + b \tag{5.30}$$

donde:

- $W \in \mathbb{R}^{k \times HWC}$ es la matriz de pesos

- $b \in \mathbb{R}^k$ es el vector de sesgos
- k es el número de neuronas en la capa

5.5.6. Aplicación de la función de activación

La salida de la capa está dada por:

$$y = \sigma(z) \tag{5.31}$$

donde σ es una función de activación aplicada componente a componente.

Interpretación

Las capas completamente conectadas pueden interpretarse como clasificadores que operan sobre las características extraídas por la red convolucional. En particular, cada neurona combina toda la información disponible en la representación vectorizada.

Esto contrasta con las capas convolucionales, donde la conectividad es local.

Clasificación

En problemas de clasificación multiclase, la última capa suele utilizar la función *softmax*, definida como:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} \tag{5.32}$$

Esta función transforma el vector z en una distribución de probabilidad sobre las clases.

Propiedades

Las capas completamente conectadas presentan las siguientes características:

- Conectividad global: cada neurona está conectada con todas las entradas
- Alta capacidad de representación
- Mayor número de parámetros en comparación con capas convolucionales

Observación

Debido al elevado número de parámetros, las capas completamente conectadas son más propensas al sobreajuste. Por esta razón, en arquitecturas modernas, su uso se reduce o se reemplaza parcialmente mediante técnicas como *global average pooling*.

5.5.7. Función de pérdida

La función de pérdida (*loss function*) cuantifica la discrepancia entre las predicciones del modelo y los valores reales. Constituye el criterio fundamental que guía el proceso de entrenamiento, ya que permite evaluar la calidad de las predicciones y definir un objetivo de optimización.

Definición general

Sea $f_\theta(x)$ la salida del modelo para una entrada x, y sea y la etiqueta real asociada. Una función de pérdida se define como:

$$\mathcal{L}(f_\theta(x), y)$$

donde $\mathcal{L} : \mathbb{R}^k \times \mathbb{R}^k \to \mathbb{R}$ mide el error entre la predicción y el valor objetivo.

Dado un conjunto de datos $\{(x_i, y_i)\}_{i=1}^N$, la función de costo global se define como:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\theta(x_i), y_i) \tag{5.33}$$

Clasificación multiclase: Entropía cruzada

En problemas de clasificación multiclase, la función de pérdida más utilizada es la entropía cruzada (*cross-entropy*).

Sea $p = f_\theta(x)$ la salida del modelo después de aplicar la función *softmax*, y sea y un vector one-hot que representa la clase verdadera. La pérdida se define como:

$$\mathcal{L}(p, y) = -\sum_{j=1}^{k} y_j \log(p_j) \tag{5.34}$$

Dado que y es un vector one-hot, esta expresión se reduce a:

$$\mathcal{L}(p, y) = -\log(p_y)$$

donde p_y es la probabilidad asignada a la clase correcta.

Interpretación probabilística

La entropía cruzada puede interpretarse como una medida de divergencia entre la distribución verdadera y y la distribución predicha p. Minimizar esta función equivale a maximizar la probabilidad de observar los datos bajo el modelo.

Desde un punto de vista estadístico, este enfoque corresponde al método de máxima verosimilitud.

Caso de regresión: Error cuadrático medio

En problemas de regresión, una función de pérdida común es el error cuadrático medio (*Mean Squared Error*, MSE), definido como:

$$\mathcal{L}(f_\theta(x), y) = \|f_\theta(x) - y\|^2 \tag{5.35}$$

Esta función penaliza de manera más severa los errores grandes, lo que favorece ajustes más precisos.

Relación con el entrenamiento

El objetivo del entrenamiento consiste en encontrar los parámetros θ que minimizan la función de costo global:

$$\theta^* = \arg\min_\theta J(\theta) \tag{5.36}$$

Este problema se resuelve típicamente mediante métodos de optimización iterativos, como el descenso por gradiente y sus variantes.

5.5.8. Entrenamiento

El entrenamiento de una red neuronal convolucional consiste en ajustar los parámetros θ del modelo con el objetivo de minimizar la función de pérdida definida previamente.

Dado un conjunto de datos $\{(x_i, y_i)\}_{i=1}^N$, el problema de entrenamiento puede formularse como:

$$\min_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f_\theta(x_i), y_i) \tag{5.37}$$

5.5.9. Método del descenso del gradiente

Considérese el problema de optimización irrestricta

$$\min_{x \in \mathbb{R}^n} f(x),$$

donde $f : \mathbb{R}^n \to \mathbb{R}$ es una función diferenciable. El objetivo consiste en construir un procedimiento iterativo que permita aproximarse a un punto minimizante de f. Entre los métodos más importantes para resolver este tipo de problemas se encuentra el método del descenso del gradiente, también conocido como *gradient descent* o método del máximo descenso.

La idea fundamental del método proviene de la interpretación geométrica del gradiente. Para una función diferenciable f, el vector gradiente

$$\nabla f(x) = \begin{pmatrix} \dfrac{\partial f}{\partial x_1}(x) \\ \dfrac{\partial f}{\partial x_2}(x) \\ \vdots \\ \dfrac{\partial f}{\partial x_n}(x) \end{pmatrix}$$

indica la dirección de máximo crecimiento local de la función en el punto x. En consecuencia, el vector opuesto

$$-\nabla f(x)$$

corresponde a la dirección de máximo decrecimiento local. El método del descenso del gradiente aprovecha precisamente esta propiedad para generar una sucesión de iterados que produzca una disminución progresiva del valor de la función objetivo.

Partiendo de un punto inicial $x_0 \in \mathbb{R}^n$, el método construye una sucesión $\{x_k\}$ definida mediante la iteración

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

donde $\alpha_k > 0$ representa el tamaño de paso asociado a la iteración k. La elección de este parámetro resulta fundamental para el desempeño del algoritmo, pues determina la magnitud del desplazamiento realizado en la dirección de descenso.

La justificación matemática del método se obtiene a partir de la noción de dirección de descenso. Sea

$$d_k = -\nabla f(x_k).$$

Entonces,

$$\nabla f(x_k)^T d_k = \nabla f(x_k)^T(-\nabla f(x_k)) = -\|\nabla f(x_k)\|^2.$$

Si $\nabla f(x_k) \neq 0$, se tiene que

$$\|\nabla f(x_k)\|^2 > 0,$$

y por tanto

$$\nabla f(x_k)^T d_k < 0.$$

Esto demuestra que d_k constituye una dirección estricta de descenso para la función f. En otras palabras, existe un valor suficientemente pequeño de $\alpha > 0$ tal que

$$f(x_k + \alpha d_k) < f(x_k).$$

Esta propiedad puede analizarse con mayor detalle mediante el desarrollo de Taylor de primer orden. Para α pequeño se tiene

$$f(x_k + \alpha d_k) = f(x_k) + \alpha \nabla f(x_k)^T d_k + o(\alpha).$$

Sustituyendo $d_k = -\nabla f(x_k)$, resulta

$$f(x_k - \alpha \nabla f(x_k)) = f(x_k) - \alpha \|\nabla f(x_k)\|^2 + o(\alpha).$$

Debido a que el término dominante es negativo, el valor de la función disminuye localmente para valores suficientemente pequeños de α. Esta observación constituye la base teórica del algoritmo.

La selección del tamaño de paso α_k es uno de los aspectos centrales del método. Una posibilidad consiste en utilizar un paso constante,

$$\alpha_k = \alpha,$$

lo cual simplifica considerablemente la implementación. Sin embargo, si el valor de α es demasiado grande, la sucesión puede divergir o presentar oscilaciones; por el contrario, si α es demasiado pequeño, la convergencia puede resultar excesivamente lenta.

Otra estrategia consiste en determinar el tamaño de paso mediante una búsqueda lineal exacta. En este caso, α_k se obtiene resolviendo el problema unidimensional

$$\alpha_k = \arg\min_{\alpha > 0} f(x_k - \alpha \nabla f(x_k)).$$

De esta manera, en cada iteración se minimiza exactamente la función sobre la recta definida por la dirección de descenso. Aunque este enfoque posee una sólida justificación teórica, en problemas de gran dimensión suele resultar costoso desde el punto de vista computacional. Por esta razón, frecuentemente se emplean búsquedas lineales inexactas basadas en criterios como las condiciones de Armijo o de Wolfe, las cuales garantizan una reducción suficiente de la función objetivo sin resolver completamente el subproblema unidimensional.

El criterio de terminación más habitual se basa en la norma del gradiente. Dado que un punto minimizante diferenciable debe satisfacer la condición necesaria de primer orden

$$\nabla f(x^*) = 0,$$

el algoritmo suele detenerse cuando

$$\|\nabla f(x_k)\| < \varepsilon,$$

para cierta tolerancia $\varepsilon > 0$.

Las propiedades de convergencia del método dependen de las características de la función objetivo. Si f es convexa y su gradiente es Lipschitz continuo, entonces la sucesión generada por el método converge hacia un mínimo global. Más aún, si f es fuertemente convexa, es decir, si existe $\mu > 0$ tal que

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) + \frac{\mu}{2}\|y - x\|^2,$$

entonces el método presenta convergencia lineal para elecciones apropiadas del tamaño de paso.

Para ilustrar el procedimiento, considérese la función cuadrática

$$f(x, y) = x^2 + y^2.$$

Su gradiente está dado por

$$\nabla f(x, y) = \begin{pmatrix} 2x \\ 2y \end{pmatrix}.$$

En consecuencia, la iteración del descenso del gradiente toma la forma

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \alpha \begin{pmatrix} 2x_k \\ 2y_k \end{pmatrix},$$

lo cual equivale a

$$x_{k+1} = (1 - 2\alpha)x_k, \qquad y_{k+1} = (1 - 2\alpha)y_k.$$

La convergencia hacia el origen ocurre siempre que

$$0 < \alpha < 1.$$

A pesar de su simplicidad y relevancia teórica, el método del descenso del gradiente posee ciertas limitaciones. En funciones mal condicionadas, la trayectoria de los iterados puede presentar oscilaciones pronunciadas y convergencia lenta, especialmente en regiones con geometría alargada o estrecha. Asimismo, en problemas no convexos el método puede converger hacia mínimos locales o puntos silla.

No obstante, el descenso del gradiente constituye la base conceptual de numerosos algoritmos modernos de optimización y aprendizaje automático, entre ellos el descenso estocástico del gradiente, los métodos con momentum, Nesterov, AdaGrad, RMSProp y Adam. Además, mantiene una estrecha relación con métodos de segundo orden como Newton y los algoritmos quasi-Newton.

Finalmente, el método admite una interpretación continua particularmente elegante. La iteración discreta puede verse como una aproximación numérica de la ecuación diferencial

$$\frac{dx(t)}{dt} = -\nabla f(x(t)),$$

conocida como flujo de gradiente (*gradient flow*). Esta ecuación describe una dinámica continua en la cual el sistema evoluciona siguiendo la dirección de máximo decrecimiento de la función energía f.

Regla de la cadena y retropropagación

El cálculo del gradiente en redes neuronales profundas se realiza mediante el algoritmo de retropropagación (*backpropagation*), el cual se basa en la aplicación sistemática de la regla de la cadena.

Sea una red expresada como:

$$f_\theta = f^{(L)} \circ \cdots \circ f^{(1)}$$

el gradiente de la pérdida respecto a los parámetros de una capa l se obtiene propagando el error desde la salida hacia las capas anteriores.

Este procedimiento permite calcular eficientemente:

$$\nabla_{\theta^{(l)}} J(\theta)$$

para cada capa l de la red.

Regularización

En el entrenamiento de redes neuronales, es común que el modelo se ajuste excesivamente a los datos de entrenamiento, fenómeno conocido como *sobreajuste* (*overfitting*). En este caso, el modelo presenta un buen desempeño en los datos de entrenamiento, pero generaliza pobremente a datos no vistos.

La regularización consiste en un conjunto de técnicas diseñadas para mejorar la capacidad de generalización del modelo.

Sobreajuste

El sobreajuste ocurre cuando el modelo aprende no solo las características relevantes del problema, sino también el ruido presente en los datos. Esto suele suceder cuando el modelo tiene una alta capacidad de representación en relación con la cantidad de datos disponibles.

Penalización de parámetros (Weight Decay)

Una estrategia común consiste en añadir un término de penalización a la función de costo:

$$J(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(f_\theta(x_i), y_i) + \lambda\|\theta\|^2 \tag{5.38}$$

donde:

- $\lambda > 0$ es el parámetro de regularización
- $\|\theta\|^2$ es la norma cuadrática de los parámetros

Este término penaliza valores grandes de los parámetros, favoreciendo modelos más simples.

5.5.10. Dropout

El *dropout* es una técnica de regularización utilizada en redes neuronales profundas cuyo objetivo principal es reducir el sobreajuste y mejorar la capacidad de generalización del modelo. La idea esencial consiste en introducir aleatoriedad durante el entrenamiento mediante la desactivación temporal de neuronas. Como consecuencia, en cada iteración se entrena efectivamente una subred distinta de la arquitectura original.

Consideremos una capa neuronal cuya entrada es un vector

$$x \in \mathbb{R}^{H\times W\times C},$$

con matriz de pesos

$$W \in \mathbb{R}^{m\times d}$$

donde hay m neuronas. Y vector de sesgos

$$b \in \mathbb{R}^m.$$

La salida usual de la capa está dada por

$$h = \phi(Wx + b).$$

El mecanismo de dropout introduce un vector aleatorio

$$r = (r_1, \dots, r_m),$$

cuyas componentes son independientes y satisfacen

$$r_i \sim \text{Bernoulli}(p).$$

Aquí, el parámetro $p \in (0, 1]$ representa la probabilidad de conservar activa una neurona durante el entrenamiento. En consecuencia, cada neurona tiene probabilidad $1 - p$ de ser anulada temporalmente.

La nueva activación de la capa se define entonces por

$$\tilde{h} = r \odot h,$$

donde $\odot$ denota el producto componente a componente. De esta forma, si $r_i = 0$, la activación correspondiente desaparece completamente en esa iteración; si $r_i = 1$, la neurona permanece activa.

En implementaciones modernas suele utilizarse la variante denominada *inverted dropout*, en la cual las activaciones se reescalan durante el entrenamiento:

$$\tilde{h} = \frac{r}{p} \odot h.$$

La razón matemática de este reescalamiento es preservar el valor esperado de las activaciones. En efecto, puesto que

$$\mathbb{E}[r_i] = p,$$

se obtiene

$$\mathbb{E}\left[\frac{r_i}{p} h_i\right] = \frac{1}{p}\mathbb{E}[r_i]h_i = h_i.$$

Por tanto, el valor esperado de las activaciones permanece invariante:

$$\mathbb{E}[\tilde{h}] = h.$$

Gracias a esta propiedad, durante la etapa de inferencia no es necesario realizar ajustes adicionales: simplemente se utiliza la red completa sin eliminar neuronas.

Desde un punto de vista probabilístico, el dropout puede interpretarse como un procedimiento que entrena simultáneamente un gran conjunto de subredes. Si una red posee n neuronas susceptibles de ser desactivadas, entonces existen potencialmente

$$2^n$$

subredes distintas. El entrenamiento explícito de todas ellas sería computacionalmente inviable; sin embargo, el dropout proporciona una aproximación eficiente a un promedio implícito sobre dicho ensamble de modelos.

La relevancia de esta idea se encuentra en que las redes neuronales poseen una enorme capacidad de representación y, por ello, pueden memorizar patrones específicos del conjunto de entrenamiento. Cuando ciertas neuronas dependen excesivamente de otras, aparecen fenómenos de *co-adaptación*. El dropout rompe estas dependencias debido a que cualquier neurona puede desaparecer aleatoriamente durante el entrenamiento. Como consecuencia, cada unidad debe aprender representaciones más independientes y robustas.

Formalmente, el entrenamiento ya no minimiza una función de pérdida determinista, sino una pérdida esperada respecto de la aleatoriedad introducida por las máscaras de dropout. Si m representa una realización particular de la máscara aleatoria y θ denota el conjunto de parámetros de la red, entonces el problema de optimización puede escribirse como

$$\min_{\theta} \ \mathbb{E}_m \left[\mathcal{L}(f(x; m, \theta), y) \right].$$

Aquí, $f(x; m, \theta)$ representa la subred inducida por la máscara m. Así, cada actualización del descenso del gradiente se realiza sobre una arquitectura ligeramente distinta.

Si el descenso del gradiente estándar actualiza parámetros mediante

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t),$$

entonces bajo dropout la actualización toma la forma

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(f(x; m_t, \theta_t), y),$$

donde m_t es una máscara aleatoria distinta en cada iteración. Este ruido adicional actúa como un mecanismo regularizador que dificulta el ajuste excesivamente preciso a los datos de entrenamiento.

En ciertos modelos simples, particularmente lineales, puede demostrarse que el dropout induce aproximadamente un efecto regularizador semejante a una penalización cuadrática sobre los pesos. En este sentido, su comportamiento se relaciona parcialmente con métodos de regularización tipo L^2, aunque el mecanismo subyacente es distinto, ya que aquí la regularización surge a partir de una perturbación aleatoria multiplicativa.

Desde una perspectiva geométrica, el dropout favorece representaciones distribuidas y estables frente a perturbaciones. Una neurona no puede depender de la presencia permanente de otras unidades, de modo que la información relevante termina siendo compartida entre múltiples componentes de la red. Esto produce modelos menos frágiles y con mejor capacidad de generalización.

Aunque arquitecturas modernas incorporan otros mecanismos de regularización, como *Batch Normalization* y conexiones residuales, el dropout continúa siendo una herramienta fundamental, especialmente en redes densas tradicionales y en problemas donde la cantidad de datos disponibles es limitada.

5.5.11. Optimización

El proceso de entrenamiento de una red neuronal implica la resolución de un problema de optimización en alta dimensión. En la práctica, este problema se aborda mediante métodos iterativos basados en gradiente.

Descenso por gradiente estocástico (SGD)

En lugar de utilizar todo el conjunto de datos en cada iteración, el *Stochastic Gradient Descent* (SGD) emplea subconjuntos (mini-batches):

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(f_\theta(x_i), y_i)$$

Este enfoque reduce el costo computacional y permite actualizaciones más frecuentes.

Mini-batch gradient descent

Una generalización del SGD consiste en utilizar lotes de tamaño intermedio:

$$\theta \leftarrow \theta - \eta \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathcal{L}(f_\theta(x_i), y_i) \tag{5.39}$$

donde B es el tamaño del lote.

Métodos adaptativos

Existen métodos de optimización que ajustan automáticamente la tasa de aprendizaje para cada parámetro. Uno de los más utilizados es *Adam*.

El algoritmo Adam combina ideas de momento y escalamiento adaptativo del gradiente. De manera simplificada, realiza actualizaciones de la forma:

$$\theta \leftarrow \theta - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

donde:

- m_t es el promedio exponencial de los gradientes
- v_t es el promedio exponencial de los gradientes al cuadrado
- ϵ es un término de estabilidad numérica

Tasa de aprendizaje

La tasa de aprendizaje η controla la magnitud de las actualizaciones. Su elección es crucial:

- valores pequeños producen convergencia lenta
- valores grandes pueden causar inestabilidad

En la práctica, se utilizan esquemas de decaimiento de la tasa de aprendizaje.

6 Base de datos

6.1. Generación y Caracterización del Conjunto de Datos

6.1.1. Construcción del Conjunto de Datos

El reconocimiento óptico de caracteres para sistemas de escritura indígenas, particularmente para el alfabeto náhuatl (Figura 4.1), presenta una limitación inicial crítica: la inexistencia de un conjunto de datos público, estandarizado y etiquetado que permita entrenar modelos de aprendizaje profundo. A diferencia de alfabetos latinos o asiáticos, que cuentan con repositorios ampliamente documentados (como MNIST o EMNIST), el dominio de los glifos náhuatl carece de recursos computacionales de código abierto.

Figura 4.1: Glifos del alfabeto náhuatl. Este sistema consta de 18 caracteres base, los cuales, para fines de manipulación computacional, se asocian a etiquetas del alfabeto latino.

6.1.2. Inexistencia de datasets públicos y justificación de la generación sintética

Ante esta carencia de una base de datos pública y estandarizada, fue necesario construir un conjunto de datos propio. La decisión de generar un dataset sintético se fundamenta en dos razones principales: 1. **Volumen de datos:** Proporcionar un número suficiente de ejemplos para evitar el sobreajuste en redes neuronales profundas. 2. **Variabilidad controlada:** Introducir deformaciones geométricas y estocásticas que permitan al modelo generalizar ante las irregularidades presentes en documentos históricos reales (desalineación, degradación de tinta, variabilidad del trazo).

6.1.3. Diseño de las plantillas base y morfología del glifo náhuatl

Inicialmente, se diseñaron de forma manual y vectorial 25 plantillas base por cada uno de los 18 caracteres. Este diseño no fue arbitrario; se basó en el análisis morfológico de los glifos, los cuales se caracterizan por una combinación de trazos curvos, segmentos rectos, puntos diacríticos y bucles cerrados. La representación base se normalizó a una cuadrícula de 28×28 píxeles, preservando la proporción y densidad de "tinta" típica de los códices, estableciendo así un punto de partida topológicamente válido para la posterior augmentación.

6.2. Transformaciones Geométricas y Estocásticas (Data Augmentation)

Para emular las condiciones de documentos, se aplicaron transformaciones a las plantillas base. A cada una de las 25 muestras iniciales se le generaron 90 variantes, resultando en un conjunto robusto y diverso.

6.2.1. Aplicación de grupos de Lie en transformaciones afines

Las transformaciones geométricas aplicadas (rotación, traslación y escalamiento) pueden formalizarse como acciones del grupo afín $Aff(2)$ sobre el espacio de la imagen. Una transformación afín se expresa como:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} s_x \cos\theta & -s_y \sin\theta \\ s_x \sin\theta & s_y \cos\theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

donde θ representa la rotación, (t_x, t_y) la traslación, y (s_x, s_y) los factores de escalamiento. En la implementación práctica, se utilizaron operadores estocásticos acotados (ej. $\theta \sim \mathcal{U}(-0.05\pi, 0.05\pi)$) para garantizar que la identidad del glifo se preservara semánticamente.

6.2.2. Modelado de ruido y degradación

Para simular la naturaleza física del papel histórico y la tinta, se inyectaron perturbaciones estocásticas: - **Ruido aditivo:** Se modeló como ruido blanco gaussiano $\mathcal{N}(0, \sigma^2)$ para emular las imperfecciones del sensor de escaneo. - **Degradación de bordes:** Se aplicó un desenfoque gaussiano leve para simular el "sangrado" de la tinta en el papel poroso.

6.2.3. Análisis estadístico del dataset resultante

El proceso generó un dataset final balanceado de $18 \times 25 \times 90 = 40,500$ imágenes. Al ser sintético, el **balance de clases** es adecuado (exactamente 2,250 muestras por clase), este tamaño es el estandard y usual al trabajar para reconocimiento de patrones. La **varianza intra-clase** fue maximizada de manera controlada mediante las transformaciones afines, asegurando que el modelo no aprenda artefactos de la plantilla base, sino las invariantes morfológicas del carácter.

6.3. Mapeo Dimensional: De $\mathbb{R}^{28\times28}$ a $\mathbb{R}^T$

La pérdida de información topológica al proyectar una imagen en una secuencia temporal.

Para abordar el reconocimiento desde una perspectiva de secuencias, es necesario proyectar la representación bidimensional de la imagen $X \in \mathbb{R}^{H\times W}$ en una secuencia unidimensional de vectores $S \in \mathbb{R}^T$. Este mapeo implica una pérdida inherente de la topología 2D (vecindades verticales y horizontales simultáneas), la cual debe ser compensada mediante la extracción de rasgos estructurales ricos en cada paso de tiempo.

6.3.1. Definición del operador de ventana deslizante vertical

Se define un operador de ventana deslizante vertical que recorre la imagen de izquierda a derecha. Para una imagen X de ancho W, se extraen T ventanas de ancho w y stride s, donde $T = \lfloor \frac{W-w}{s} \rfloor + 1$. Cada elemento de la secuencia $S_t \in \mathbb{R}^{H\times w}$ representa una "rebanada" vertical de la imagen en el paso de tiempo t, preservando la estructura vertical del trazo en ese instante.

6.4. Extracción Matemática de Rasgos Estructurales (Feature Engineering)

Para mitigar la pérdida topológica mencionada, se extraen descriptores matemáticos de cada ventana S_t.

6.4.1. Cálculo del Centro de Gravedad

El primer momento de la distribución de "tinta" (intensidad de píxeles) proporciona el centroide local de la masa del trazo. Para una ventana S_t, las coordenadas del centro de gravedad (μ_x, μ_y) se calculan como:

$$\mu_x = \frac{\sum_{i,j} j \cdot S_t(i,j)}{\sum_{i,j} S_t(i,j)}, \quad \mu_y = \frac{\sum_{i,j} i \cdot S_t(i,j)}{\sum_{i,j} S_t(i,j)}$$

Este rasgo es invariante a traslaciones globales y altamente discriminativo para la morfología del glifo.

6.4.2. Transiciones de estado y cruces por cero

Se analiza la derivada discreta del perfil de proyección horizontal de la ventana. Los cruces por cero de esta derivada indican la presencia de múltiples trazos horizontales superpuestos o la transición entre el fondo y la tinta, proporcionando información sobre la complejidad estructural local.

6.4.3. Perfiles superior e inferior (Envoltura convexa)

Se calculan los perfiles superior P_{sup} e inferior P_{inf} de la ventana, definidos como los índices de fila mínimo y máximo, respectivamente, donde la intensidad del píxel supera un umbral τ. La diferencia $P_{sup} - P_{inf}$ define el espesor local del trazo, actuando como una aproximación discreta de la envoltura convexa del carácter en esa columna.

6.5. Cuantificación y Discretización del Espacio de Observaciones

Para alimentar modelos que requieren espacios de observación discretos, el espacio continuo de rasgos debe ser discretizado.

6.5.1. Función de cuantificación por intervalos (Binning)

Dado un rasgo continuo $v \in [v_{min}, v_{max}]$, se aplica una función de cuantificación uniforme que lo mapea a uno de N intervalos (bins):

$$q(v) = \left\lfloor \frac{v - v_{min}}{v_{max} - v_{min}} \times N \right\rfloor$$

6.5.2. Codificación binaria y construcción del alfabeto de 256 símbolos observables

Combinando los rasgos cuantificados (por ejemplo, centroide Y, espesor del trazo y perfil superior), se construye un vector de características que se codifica en una representación binaria de 8 bits. Esto permite definir un **alfabeto discreto de exactamente 256 símbolos observables** (2^8), donde cada símbolo representa una configuración estructural local única de la ventana deslizante.

6.5.3. Análisis de la entropía de Shannon de las secuencias generadas

La riqueza informativa de las secuencias generadas se valida mediante la entropía de Shannon $H(S)$ de la distribución de símbolos en el dataset:

$$H(S) = -\sum_{i=1}^{256} p(s_i) \log_2 p(s_i)$$

Una entropía alta indica que el proceso de generación sintética y extracción de rasgos produce una diversidad de patrones locales rica y no redundante, lo cual es un prerrequisito fundamental para que un modelo de aprendizaje automático pueda aprender representaciones discriminativas y generalizar adecuadamente.

(Nota: El código de carga de datos en TensorFlow que tenías originalmente es totalmente válido y se puede insertar como una subsección práctica al final, por ejemplo, bajo un "4.6. Implementación Computacional del Pipeline", si tu director lo permite, o integrarlo como ejemplos dentro de las secciones anteriores).

6.6. Construcción del conjunto de datos

El reconocimiento óptico de caracteres para sistemas de escritura indígenas, particularmente para el alfabeto Náhuatl Figura 6.1, presenta una limitación inicial importante: no existe un conjunto de datos público, estandarizado y etiquetado que permita entrenar modelos de aprendizaje profundo. A diferencia de otros alfabetos, para los cuales existen repositorios ampliamente documentados, el alfabeto Náhuatl carece de recursos computacionales que puedan emplearse directamente en tareas de clasificación automática.

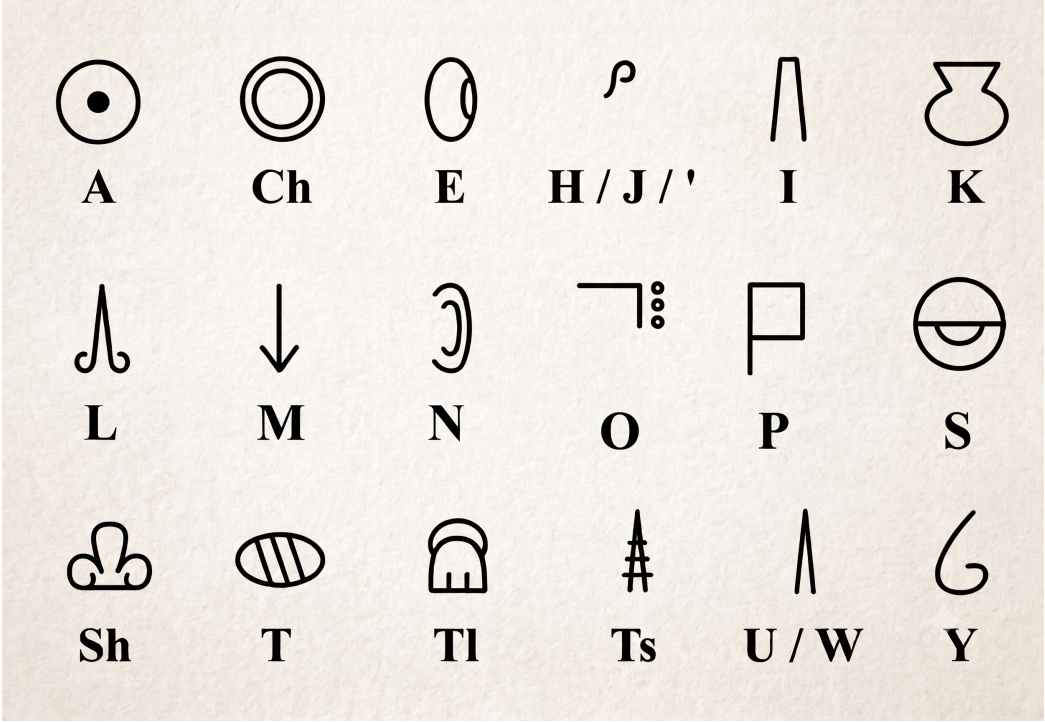


Figura 6.1: Glifos del alfabeto nahuatl. Este alfabeto consta de 18 caracteres, para los cuales se emplean letras del alfabeto latino para su manipulación.

Ante esta situación, fue necesario construir un conjunto de datos propio. La decisión de generar un dataset sintético se tomó por dos razones principales:

1. Proporcionar un número suficiente de ejemplos para el entrenamiento de la red neuronal.

2. Introducir variaciones geométricas que permitan al modelo generalizar ante deformaciones presentes en documentos reales.

6.6.1. Generación de las muestras

Inicialmente se diseñaron de forma manual 25 ejemplos base para cada carácter del alfabeto. Posteriormente, a cada muestra se le aplicaron transformaciones geométricas y perturbaciones aleatorias para incrementar la variabilidad del conjunto de datos.

La Figura Figura 6.2 muestra algunos ejemplos de las imágenes generadas.

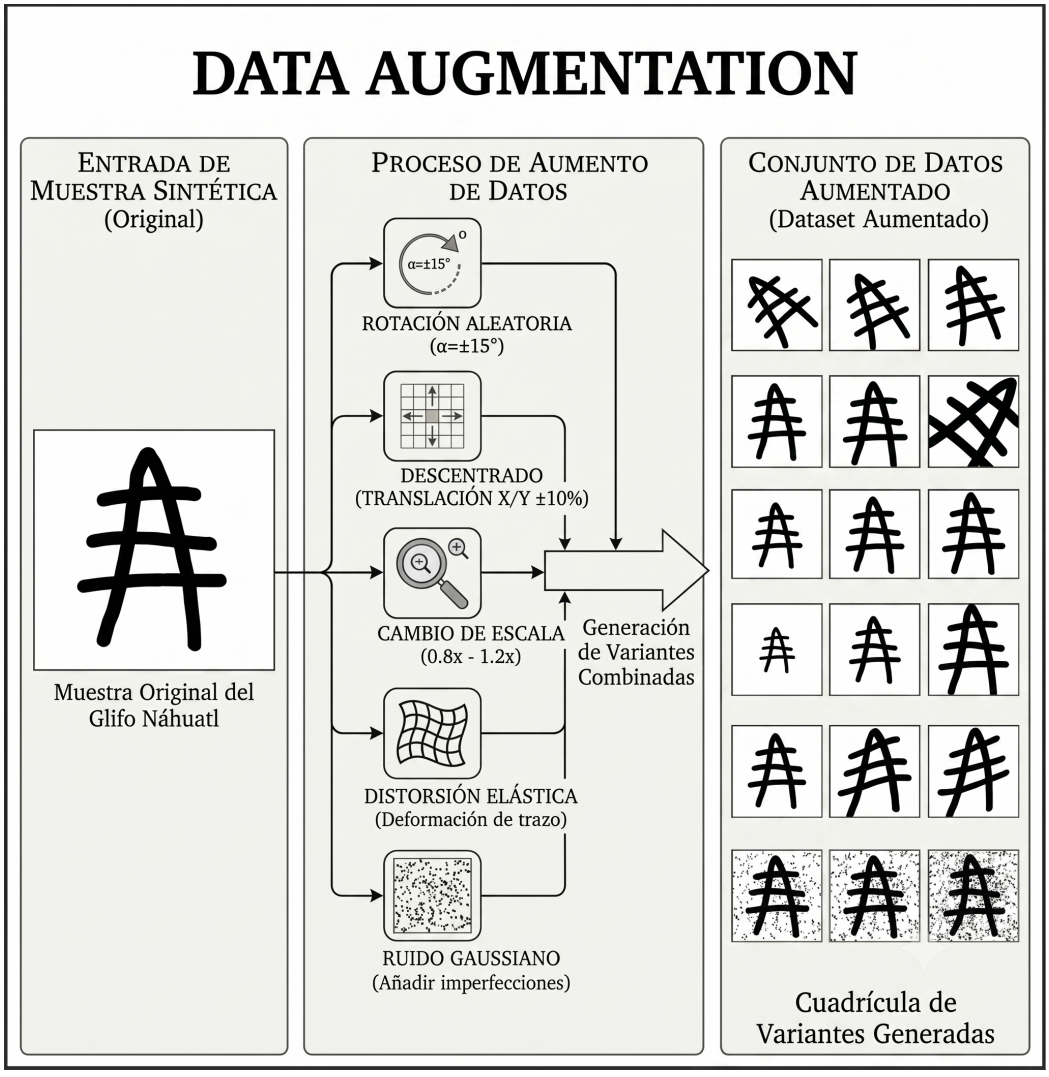


Figura 6.2: Se observa el proceso por el que pasa cada una de las 25 imagenes por letra para generar 90 variantes obtenidas a partir de rotación, traslación, distorción elástica, cambio de escala y ruido, de este modo se consigue un dataset más completo.

6.7. Carga y organización de los datos

Una vez construido el conjunto de datos, las imágenes se almacenaron siguiendo una estructura jerárquica de directorios, donde cada carpeta representa una clase distinta del alfabeto.

El cargado de las imágenes se realizó mediante la función `image_dataset_from_directory` de TensorFlow:

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="training",
    seed=123,
    image_size=(28,28),
    batch_size=32,
    color_mode="grayscale"
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(28,28),
    batch_size=32,
    color_mode="grayscale"
)
```

La elección de una partición de entrenamiento-validación de 80 %-20 % constituye una práctica común en aprendizaje automático, ya que permite disponer de un número suficiente de ejemplos para el ajuste de los parámetros del modelo y, al mismo tiempo, reservar un conjunto independiente para evaluar la capacidad de generalización.

Asimismo, las imágenes se convirtieron a escala de grises. Dado que los glifos del alfabeto Náhuatl se distinguen principalmente por su geometría y no por información cromática, el uso de un solo canal reduce el costo computacional sin sacrificar información relevante.

6.8. Normalización y aumentación de datos

Antes de alimentar las imágenes a la red neuronal, se realizó un proceso de normalización:

```
normalization_layer = tf.keras.layers.Rescaling(1./255)
```

Esta transformación reescala los valores de intensidad al intervalo $[0, 1]$, lo cual mejora la estabilidad numérica del proceso de entrenamiento y facilita la convergencia del algoritmo de optimización.

Adicionalmente, se implementó un mecanismo de aumentación de datos en línea:

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomRotation(0.05),
    tf.keras.layers.RandomZoom(0.1),
    tf.keras.layers.RandomTranslation(0.05, 0.05)
])
```

La introducción de pequeñas rotaciones, cambios de escala y traslaciones tiene como objetivo simular las variaciones presentes en documentos escaneados o en la escritura manual. De esta manera, el modelo aprende representaciones más robustas y reduce su tendencia al sobreajuste.

Finalmente, los datos fueron almacenados temporalmente en memoria y preparados para alimentar la GPU:

```
train_ds = train_ds.cache().shuffle(5000).prefetch(AUTOTUNE)
val_ds = val_ds.cache().prefetch(AUTOTUNE)
```

El uso de `cache()` evita que las imágenes sean leídas repetidamente desde el disco, mientras que `prefetch()` permite superponer las operaciones de lectura de datos con el entrenamiento de la red, disminuyendo el tiempo total de ejecución.

7 Caso de estudio

7.1. CASO CNN

7.1.1. Arquitectura de la red neuronal convolucional

La arquitectura propuesta recibe imágenes de tamaño $28 \times 28 \times 1$ y está compuesta por tres bloques convolucionales seguidos de un clasificador denso.

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28,28,1)),

    tf.keras.layers.Conv2D(32,3,activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.SpatialDropout2D(0.25),

    tf.keras.layers.Conv2D(64,3,activation='relu'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.SpatialDropout2D(0.25),

    tf.keras.layers.Conv2D(128,3,activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.SpatialDropout2D(0.25),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes, activation='softmax')
])
```

7.1.2. Primer bloque convolucional

La primera capa convolucional emplea 32 filtros de tamaño 3×3:

```
tf.keras.layers.Conv2D(32,3,activation='relu')
```

Este bloque se encarga de detectar características de bajo nivel, tales como bordes, esquinas y pequeños segmentos de los trazos.

La incorporación de una capa de normalización por lotes:

```
tf.keras.layers.BatchNormalization()
```

permite estabilizar la distribución de las activaciones internas, acelerando el proceso de convergencia.

Posteriormente, la operación de *max pooling*

```
tf.keras.layers.MaxPooling2D()
```

reduce la dimensionalidad espacial de las representaciones, disminuyendo el número de parámetros y haciendo al modelo menos sensible a pequeñas traslaciones.

Finalmente, se incorpora un mecanismo de regularización:

```
tf.keras.layers.SpatialDropout2D(0.25)
```

que desactiva mapas de características completos de manera aleatoria, obligando a la red a aprender representaciones más robustas.

7.1.3. Segundo y tercer bloque convolucional

Los bloques posteriores incrementan progresivamente el número de filtros hasta 64 y 128, respectivamente. El aumento en la profundidad de la red permite capturar patrones cada vez más complejos, tales como combinaciones de curvas, intersecciones y configuraciones geométricas características de cada glifo.

7.1.4. Clasificador

Las características extraídas se transforman en un vector unidimensional:

```
tf.keras.layers.Flatten()
```

y posteriormente se procesan mediante una capa densa:

```
tf.keras.layers.Dense(256, activation='relu')
```

La capa de salida utiliza la función Softmax:

```
tf.keras.layers.Dense(num_classes, activation='softmax')
```

la cual produce una distribución de probabilidad sobre todas las clases del alfabeto.

7.2. Entrenamiento

El modelo se compiló utilizando el optimizador Adam:

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

La función de pérdida de entropía cruzada dispersa se seleccionó debido a que las etiquetas de las imágenes se encuentran codificadas como enteros y no mediante vectores *one-hot*.

Para evitar el sobreajuste se implementó un mecanismo de detención temprana:

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=7,
    restore_best_weights=True
)
```

La elección de siete épocas de paciencia permite al modelo continuar el aprendizaje ante pequeñas oscilaciones de la pérdida de validación, evitando finalizar el entrenamiento de forma prematura.

Asimismo, el mejor modelo obtenido se almacenó automáticamente:

```
checkpoint = ModelCheckpoint(
    ruta_modelo,
    monitor='val_loss',
    save_best_only=True
)
```

7.3. Sistema de inferencia interactivo

Finalmente, se desarrolló una interfaz interactiva en Google Colab que permite dibujar caracteres del alfabeto Náhuatl y obtener predicciones en tiempo real.

La imagen generada por el usuario se redimensiona a 28×28 píxeles:

```
img_resized = cv2.resize(img, (28,28))
img_normalized = img_resized / 255.0
img_input = np.expand_dims(img_normalized, axis=(0,-1))
```

Posteriormente, el modelo produce un vector de probabilidades:

```
predicciones = model.predict(img_input)
```

y se determina la clase más probable mediante:

```
idx_ganador = np.argmax(predicciones)
```

Además de la predicción principal, la interfaz presenta las tres clases con mayor probabilidad, proporcionando información adicional sobre la confianza y las posibles ambigüedades del modelo.

7.4. CASO HMM

7.5. CASO HMM

En este capítulo se describe la metodología implementada para el reconocimiento automático de letras mayúsculas manuscritas del alfabeto latino mediante Modelos Ocultos de Markov (HMM). A diferencia del capítulo anterior, los bloques de código se muestran con fines documentales y no son ejecutados automáticamente dentro del documento debido al alto costo computacional del entrenamiento.

7.5.1. Configuración del entorno experimental

El primer paso consistió en importar las bibliotecas necesarias para el procesamiento de imágenes, manipulación de datos, entrenamiento de los modelos HMM y evaluación de resultados, estableciendo una conexión con un sistema de almacenamiento externo:

```
from google.colab import drive
drive.mount('/content/drive')
import cv2
import numpy as np
import torchvision.datasets as datasets
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import seaborn as sns
import random
```

```
import os
import pickle
from tqdm import tqdm
try:
    from hmmlearn import hmm
except ImportError:
    import subprocess
    import sys
    subprocess.check_call([sys.executable, "-m", "pip", "install", "hmmlearn"])
    from hmmlearn import hmm
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import warnings
```

Para asegurar la reproducibilidad de los resultados, se fijaron semillas aleatorias y se deshabilitaron advertencias no críticas:

```
np.random.seed(42)
random.seed(42)
warnings.filterwarnings("ignore")
```

7.5.2. Definición de parámetros del experimento

A continuación, se definieron los parámetros principales utilizados durante el entrenamiento y la evaluación del sistema:

```
N_ESTADOS = 12
TRAIN_POR_CLASE = 1500
TEST_POR_CLASE = 400

IMG_SIZE = 40
UMBRAL = 0.30

ARCHIVO_MODELOS_CAT = "/content/drive/MyDrive/modelos_categorical_v_avanzada_hmm.pkl"

TARGETS_MAYUSCULAS = list(range(10,36))
MAPEO_LETTERS = {i: chr(ord('A') + (i - 10)) for i in range(10,36)}
LETRAS_OBJETIVO = [chr(ord('A') + i) for i in range(26)]
```

Se optó por utilizar 12 estados ocultos para equilibrar la capacidad descriptiva y el costo computacional, seleccionando 1500 muestras de entrenamiento y 400 de prueba por clase con un tamaño de imagen fijado en 40×40 píxeles.

7.5.3. Obtención del conjunto de datos

Para el entrenamiento y evaluación del sistema se empleó el conjunto de datos EMNIST ByClass:

```
print("Descargando EMNIST ByClass...")
emnist_train = datasets.EMNIST(root='./data', split='byclass',
                               train=True, download=True,
                               transform=transforms.ToTensor())
emnist_test = datasets.EMNIST(root='./data', split='byclass',
                               train=False, download=True,
                               transform=transforms.ToTensor())
```

De este conjunto se seleccionaron exclusivamente las letras mayúsculas con el propósito de acotar el problema de clasificación.

7.5.4. Arquitectura del HMM

Se construyó una matriz de transición tipo Bakis flexible para modelar la evolución progresiva de los trazos:

```
def construir_matriz_bakis_flexible(n_states):
    A = np.zeros((n_states, n_states))
    for i in range(n_states):
        if i == n_states - 1:
            A[i, i] = 1.0
        elif i == n_states - 2:
            A[i, i] = 0.60
            A[i, i+1] = 0.40
        else:
            A[i, i] = 0.60
            A[i, i+1] = 0.30
            A[i, i+2] = 0.10
    return A
```

La estructura Bakis impone una restricción direccional que permite permanecer en el mismo estado o avanzar, evitando retrocesos, lo cual resulta adecuado para describir procesos secuenciales como la formación de trazos manuscritos.

7.5.5. Preprocesamiento de imágenes

Antes de convertir las imágenes en secuencias observables, se aplicó un proceso de normalización y alineamiento:

```
def procesar_imagen(imagen_tensor):
    imagen = imagen_tensor.squeeze().numpy()
    imagen = np.fliplr(imagen)
    imagen = np.rot90(imagen)

    img_bin = (imagen > UMBRAL).astype(np.uint8)
    filas = np.any(img_bin, axis=1)
    columnas = np.any(img_bin, axis=0)

    if np.any(filas) and np.any(columnas):
        ymin, ymax = np.where(filas)[0][[0,-1]]
        xmin, xmax = np.where(columnas)[0][[0,-1]]
        recorte = imagen[ymin:ymax+1, xmin:xmax+1]

        h, w = recorte.shape
        max_dim = max(h, w)
        pad_h = max_dim - h
        pad_w = max_dim - w

        pad_top = pad_h // 2
        pad_bottom = pad_h - pad_top
        pad_left = pad_w // 2
        pad_right = pad_w - pad_left

        recorte_cuadrado = np.pad(recorte, ((pad_top, pad_bottom),
        (pad_left, pad_right)),
        mode='constant', constant_values=0)
        imagen = cv2.resize(recorte_cuadrado.astype(np.float32),
        (IMG_SIZE, IMG_SIZE),
        interpolation=cv2.INTER_LINEAR)
    else:
        imagen = cv2.resize(imagen.astype(np.float32), (IMG_SIZE, IMG_SIZE),
        interpolation=cv2.INTER_LINEAR)

    imagen = (imagen > UMBRAL).astype(float)
    return imagen
```

Este procedimiento incluyó la corrección de orientación del conjunto EMNIST, la eliminación de espacios vacíos mediante recortes, relleno para conservar proporciones y redimensionamiento uniforme.

7.5.6. Extracción de características

Cada imagen preprocesada fue transformada en una secuencia discreta de observaciones mediante ventanas deslizantes verticales:

```
def imagen_a_secuencia(img, w=5, stride=2):
    secuencia = []
    altura = IMG_SIZE

    for t in range(0, img.shape[1] - w + 1, stride):
        ventana = img[:, t:t+w]
        columna = np.mean(ventana, axis=1)
        col_bin = (columna > 0.35).astype(int)

        suma_tinta = np.sum(col_bin)

        if suma_tinta == 0:
            simbolo = 0
            secuencia.append([simbolo])
            continue

        if suma_tinta < 8: tinta_q = 0
        elif suma_tinta < 16: tinta_q = 1
        elif suma_tinta < 24: tinta_q = 2
        else: tinta_q = 3

        cg = np.sum(col_bin * np.arange(altura)) / suma_tinta
        if cg < 12: cg_q = 0
        elif cg < 20: cg_q = 1
        elif cg < 28: cg_q = 2
        else: cg_q = 3

        transiciones = np.sum(np.abs(np.diff(col_bin)))
        trans_q = min(int(transiciones), 3)

        indices_tinta = np.where(col_bin > 0)[0]
        perfil_sup = indices_tinta[0]
        perfil_inf = indices_tinta[-1]

        up_q = 1 if perfil_sup > 10 else 0
        low_q = 1 if perfil_inf < 30 else 0

        simbolo = (tinta_q << 6) | (cg_q << 4) | (trans_q << 2) | (up_q << 1) | low_q
        secuencia.append([simbolo])

    return np.array(secuencia)
```

Se calcularon características geométricas (distribución de tinta, centro de gravedad, número de transiciones y perfiles superior e inferior), cuantificadas en un conjunto discreto de 256 símbolos observables.

7.5.7. Construcción de los conjuntos de entrenamiento y prueba

Las secuencias obtenidas fueron organizadas en conjuntos balanceados de entrenamiento y evaluación:

```
def construir_dataset(dataset, max_muestras):
    X, y = [], []
    conteos = {k:0 for k in TARGETS_MAYUSCULAS}

    for tensor, target in dataset:
        if target not in TARGETS_MAYUSCULAS: continue
        if conteos[target] >= max_muestras: continue

        img = procesar_imagen(tensor)
        seq = imagen_a_secuencia(img)

        X.append(seq)
        y.append(MAPEO_LETTERS[target])
        conteos[target] += 1

        if all(c >= max_muestras for c in conteos.values()): break

    return X, np.array(y)

print("Construyendo entrenamiento...")
X_train, y_train = construir_dataset(emnist_train, TRAIN_POR_CLASE)

print("Construyendo prueba...")
X_test, y_test = construir_dataset(emnist_test, TEST_POR_CLASE)
```

7.5.8. Entrenamiento de los modelos

Se entrenó un modelo HMM independiente para cada una de las letras del alfabeto:

```
modelos_hmm = {}

if os.path.exists(ARCHIVO_MODELOS_CAT):
    print("\n[INFO] Modelos detectados en Drive. Cargando...")
```

```
    with open(ARCHIVO_MODELOS_CAT, 'rb') as f:
        modelos_hmm = pickle.load(f)
else:
    print("\nEntrenando CategoricalHMMs...\n")
    barra = tqdm(LETRAS_OBJETIVO, desc="Entrenando", unit="letra")
    for letra in barra:
        barra.set_description(f"Entrenando '{letra}'")
        idx = np.where(y_train == letra)[0]
        secuencias = [X_train[i] for i in idx]
        lengths = [len(s) for s in secuencias]
        X_concat = np.vstack(secuencias)

        model = hmm.CategoricalHMM(
            n_components=N_ESTADOS,
            n_features=256,
            n_iter=250,
            random_state=42,
            init_params='e'
        )

        model.startprob_ = np.zeros(N_ESTADOS)
        model.startprob_[0] = 1.0
        model.transmat_ = construir_matriz_bakis_flexible(N_ESTADOS)

        model.fit(X_concat, lengths)
        modelos_hmm[letra] = model

    with open(ARCHIVO_MODELOS_CAT, 'wb') as f:
        pickle.dump(modelos_hmm, f)
```

7.5.9. Inferencia y clasificación

Cada muestra del conjunto de prueba fue evaluada frente a todos los HMM disponibles:

```
print("\nClasificando...\n")
y_pred = []

for seq in tqdm(X_test, desc="Evaluando Inferencia"):
    scores = {}
    for letra, model in modelos_hmm.items():
        try: scores[letra] = model.score(seq)
        except: scores[letra] = -np.inf
```

```
    pred = max(scores, key=scores.get)
    y_pred.append(pred)
```

La predicción final se obtuvo seleccionando la letra cuyo modelo produjo la mayor puntuación de verosimilitud (principio de máxima verosimilitud).

7.5.10. Evaluación del desempeño

Finalmente, se calcularon métricas globales e individuales de rendimiento para evaluar la capacidad de generalización del sistema:

```
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred, labels=LETRAS_OBJETIVO)

precision_por_letra = cm.diagonal() / cm.sum(axis=1)

print("\n" + "="*70)
print("EXACTITUD (ACCURACY) INDIVIDUAL POR LETRA")
print("="*70)
for i, letra in enumerate(LETRAS_OBJETIVO):
    acc = precision_por_letra[i] * 100
    print(f"Precisión Letra {letra} : {acc:>6.2f}%")

print("\n" + "="*70)
print(f"ACCURACY GLOBAL: {accuracy*100:.2f}%")
print("="*70 + "\n")

print(classification_report(y_test, y_pred, target_names=LETRAS_OBJETIVO))
```

Asimismo, se construyó una matriz de confusión para identificar visualmente los patrones de error entre caracteres similares:

```
plt.figure(figsize=(16,14))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=LETRAS_OBJETIVO,
yticklabels=LETRAS_OBJETIVO)
plt.title(f'Matriz CategoricalHMM con Rasgos Estructurales (Precisión: {accuracy*100:.2f}%
plt.ylabel('Letra Real', fontsize=14)
plt.xlabel('Letra Predicha', fontsize=14)
plt.tight_layout()
plt.show()
```

8 Conclusiones: Comparación entre HMM y CNN en Reconocimiento de Caracteres

8.1. Conclusiones

La comparación entre los modelos ocultos de Markov (HMM) y las redes neuronales convolucionales (CNN) en el reconocimiento de caracteres evidencia un cambio de paradigma fundamental en el procesamiento de imágenes. La diferencia más significativa radica en el proceso de extracción de características. Mientras que los HMM dependen de un diseño manual y heurístico de los rasgos geométricos o estructurales, lo que limita su capacidad de adaptación, las CNN operan bajo un enfoque de extremo a extremo. Esto les permite aprender de manera automática y jerárquica las representaciones espaciales directamente desde los píxeles, eliminando el sesgo humano y capturando patrones visuales mucho más complejos.

Esta distinción arquitectónica se traduce directamente en un rendimiento superior de las redes convolucionales en términos de precisión y robustez. Las CNN manejan con mucha mayor eficacia la variabilidad inherente a los caracteres, como las diferencias tipográficas, el ruido en la imagen o las deformaciones en la escritura a mano alzada. Aunque los HMM poseen una ventaja teórica natural para el modelado de datos secuenciales, su desempeño en caracteres aislados o en entornos con alta variabilidad espacial se estanca rápidamente. Por el contrario, la capacidad de las CNN para procesar estructuras bidimensionales las consolida como la arquitectura dominante para la extracción de rasgos visuales precisos.

No obstante, esta superioridad en precisión conlleva un mayor costo computacional y una dependencia crítica de grandes volúmenes de datos etiquetados para evitar el sobreajuste. Los HMM, al ser modelos paramétricos más simples, siguen siendo una alternativa viable en escenarios donde los recursos de hardware son limitados o los conjuntos de datos de entrenamiento son reducidos. En conclusión, aunque los modelos ocultos de Markov sentaron las bases estadísticas para el reconocimiento óptico, las redes neuronales convolucionales representan el estándar actual e insustituible. La elección entre ambas metodologías ya no responde a una preferencia teórica, sino a la disponibilidad de datos y capacidad de cómputo, siendo las CNN la opción definitiva para cualquier aplicación moderna que requiera alta precisión y escalabilidad.

9 Referencias